



MCT 343 – Intro to biomechatronics

Final Report

Fall 2025

Team (7)

Submitted to:

Dr. Mohammed Ibrahim Awad

Eng. Mohamed Mohanna

Eng. Hamdy Osama

Prepared by:

Contribution

Kenzy Ahmed Abd El Moneam	(22P0107)	27 %
Ahmed Mohamed Elbehiry	(22P0044)	27 %
Ahmed Hisham Mohamed	(2200618)	27 %
Philopatir Samir Talaat	(22P0205)	19 %

Contents

List of figures	ii
Abstract	iii
1. Introduction	1
2. Conversion from (.json) to a structured (.csv) file for data handling.....	1
3. Signals Preprocessing.....	6
3.1. Bandpass Filtering (10–500 Hz)	6
3.2. Full-Wave Rectification	7
3.3. Normalization (Scaling to Maximum Value).....	7
4. Time-domain Analyses.....	8
4.1. Mean Absolute Value (MAV).....	8
4.2. Root Mean Square (RMS).....	9
4.3. Waveform Length (WL)	9
4.4. Integrated EMG (iEMG).....	9
4.5. Slope Sign Changes (SSC).....	9
4.6. Hurst Component (Hurst Exponent)	10
Why RMS and iEMG Are the Best Metrics for Dominant Channel Selection.....	11
5. Frequency-domain Analyses	11
5.1. Converting EMG to the Frequency Domain (FFT).....	11
5.2. Power Spectral Density (PSD)	12
6. Time-frequency Analyses.....	13
6.1. Discrete Wavelet Transform (DWT).....	13
Interpretation of the Provided DWT Plots.....	14
7. Dominant Channel Selection.....	15
6.1. First gesture: Index Contraction/ Relaxation.....	16
Justification for Using Time-Domain Features Instead of Frequency-Domain or Time-Frequency Methods.....	16
6.1.1. Conclusion of Channel Selection for index.....	17
6.1.1.1. Some metrics for the two dominant channels for the index.....	18
6.2. Second gesture: Pinky Contraction/Relaxation	23
6.2.1. Conclusion of Channel Selection for pinky finger.....	23
6.2.1.1. Some metrics for channel 6 for the pinky.....	24
CAD implementation and testing using a dummy signal on MATLAB.....	27
7. Flowchart for Control Algorithm	29
7.1. Control Algorithm Description	29
7.1.1. System Initialization	29



7.1.2.	EMG Signal Input and Processing.....	30
7.1.3.	Threshold Comparison and State Selection	
7.1.4.	Motion Smoothing and Actuation.....	31
7.1.5.	Continuous Feedback Loop	32
8.	Final MATLAB model	32
9.	Conclusion.....	36
10.	Submission link.....	36
11.	Appendix	37
11.1.	Conversion from .json to a structured .csv for better file handling.....	37
11.2.	Preprocessing MATLAB script.....	39
11.3.	Time-domain analyses script.....	42
11.4.	Frequency-domain analyses script.....	45
11.5.	Frequency-time analysis script.....	50
11.6.	RMS envelope for weighted averaged signal.....	52
11.7.	Average dominant channels	52
11.8.	Hurst Component Formulas for calculations.....	53

List of figures:

Figure 1-	EMG signal data	1
Figure 2-	Processed data	2
Figure 3-	Raw EMG signal of channel 14 of pinky	6
Figure 4-	Filtered EMG signal for channel 14 of pinky	7
Figure 5-	Rectified EMG signal for channel 14 of pinky.....	7
Figure 6-	Normalized EMG signal for channel 14 of pinky	8
Figure 7-	Time-domain analyses for processed EMG signal channel 14 pinky	10
Figure 8-	FFT for channel 14 of pinky.....	12
Figure 9-	PSD for channel 14 of pinky.....	13
Figure 10-	DWT plots for channel 14 of pinky.....	14
Figure 11	Weighted RMS EMG signal of index	18
Figure 12-	Time domain metrics for channel 5 of Index finger	18
Figure 13-	Time domain metrics for channel 8 of Index finger	19
Figure 14-	Hurst Component for index dominant channels.....	19
Figure 15-	FFT for channel 5 of index finger	19
Figure 16-	FFT for channel 8 of index finger	20
Figure 17-	PSD for channel 5 of index finger.....	20
Figure 18-	PSD for channel 8 of index finger.....	21
Figure 19-	PSD band power 20-450Hz for index finger.....	21
Figure 20-	DWT for channel 5 of index finger.....	21
Figure 21-	DWT for channel 8 of index finger.....	22



Figure 22- Summation for absolute D3 & D4 for index dominant channels	22
Figure 23- Weighted RMS EMG signal for pinky	24
Figure 24- Time domain metrics for channel 6 of pinky finger	24
Figure 25- Hurst Component for dominant channels of pinky	25
Figure 26- FFT for channel 6 of pinky finger	25
Figure 27- PSD for channel 6 of pinky finger	25
Figure 28- PSD band power 20-450Hz for pinky finger	26
Figure 29- DWT for channel 6 of pinky finger	26
Figure 30- Summation for absolute D3 & D4 for pinky dominant channels	26
Figure 31- Snippet of MATLAB sim to test joints' motion	27
Figure 32- Prosthetic hand relaxed	28
Figure 33- Prosthetic hand contracted	28
Figure 34- flowchart for control algorithm	29
Figure 35- MATLAB model	32
Figure 36- Controller subsystem model on MATLAB	33

Abstract

This report presents a complete workflow for the acquisition, processing, analysis, and application of surface electromyography (EMG) signals for gesture-based hand control. The study begins with the conversion of raw EMG data from JSON to structured CSV format using a custom Python script, enabling efficient data handling and analysis. The EMG signals are then pre-processed through bandpass filtering to suppress noise and motion artifacts, followed by rectification and normalization to ensure amplitude consistency across channels.

Time-domain feature extraction is performed using wavelength, integrated EMG (IEMG), slope sign changes (SSC), and root mean square (RMS), providing quantitative measures of muscle activation. Frequency-domain analysis is conducted using the Fast Fourier Transform (FFT) and Power Spectral Density (PSD) estimation via Welch's method to characterize the spectral content of the signals. To capture both temporal and spectral characteristics, a time–frequency analysis is carried out using the Discrete Wavelet Transform (DWT).

Based on IEMG and RMS metrics, dominant EMG channels are identified for two distinct gestures: index finger and pinky finger activation. These channels are combined using a weighted averaging strategy to generate representative control signals. The processed signals are further refined using an RMS sliding window envelope and applied as inputs to a MATLAB/Simulink hand model. A threshold-based control strategy with proportional control is implemented to achieve smooth transitions between open and strong grip states. The results demonstrate the effectiveness of the proposed pipeline for translating EMG activity into stable and intuitive hand motion control.

1. Introduction

Surface electromyography (EMG) provides a direct and non-invasive means of capturing muscle activation, making it a widely used signal source for gesture recognition and human-machine interaction. However, raw EMG data are inherently noisy, highly variable, and difficult to interpret without systematic processing and analysis. Reliable extraction of control-relevant information therefore requires a structured pipeline that spans data formatting, signal conditioning, feature extraction, and model-based validation.

This report presents an end-to-end EMG processing and simulation framework aimed at controlling finger-specific hand motions. The study begins with the conversion of raw EMG recordings from JSON format into structured CSV files using Python, enabling consistent downstream processing. The signals are then conditioned through bandpass filtering, rectification, and normalization to improve signal quality and ensure comparability across channels.

A comprehensive analysis is performed in the time domain, frequency domain, and time-frequency domain to capture different aspects of muscle behavior. Time-domain features such as wavelength, integrated EMG (IEMG), slope sign changes (SSC), and root mean square (RMS) are extracted to quantify muscle activation intensity and variability. Frequency-domain characteristics are examined using the Fast Fourier Transform (FFT) and power spectral density estimation via Welch's method, while the Discrete Wavelet Transform (DWT) is employed to preserve both temporal and spectral information.

Based on the extracted features, dominant EMG channels are identified for two specific gestures: index finger and pinky finger activation. These channels are combined and mapped to a MATLAB/Simulink hand model, where a threshold-based proportional control strategy is implemented to drive finger motion. By integrating signal processing with dynamic simulation, this work aims to demonstrate a coherent and practical approach for translating EMG activity into controlled hand movements.

2. Conversion from (.json) to a structured (.csv) file for data handling

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	Time	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai	Trigno Avai
2	0	-16.9528	-5.956	-7.34275	-3.76758	-1.82927	0.487805	-2.98781	0.167849	-7.16199	-6.67796	1.367849	-3.65854	-11.2195	2.012195	0.503548	-1.72018	-6.91074	-7.93852	8.780488	15.97561	1.768293	11.74945
3	0.000794	-16.6171	-5.95496	-7.34352	-3.7663	-1.82927	0.487805	-2.98781	-1.34279	-7.16151	-6.67748	1.368736	-3.65854	-11.2195	2.012195	1.007096	-1.71949	-6.91185	-7.93822	8.780488	15.97561	1.768293	20.47761
4	0.001588	-17.6242	-5.95389	-7.3443	-3.76903	-1.82927	0.487805	-2.98781	-2.01419	-7.161	-6.67702	1.369595	-3.65854	-11.2195	2.012195	-0.50355	-1.71877	-6.91299	-7.93792	8.780488	15.97561	1.768293	21.31685
5	0.002382	-10.9102	-5.95278	-7.3451	-3.76978	-1.82927	0.487805	-2.98781	-0.3357	-7.16048	-6.67659	1.370423	-3.65854	-11.2195	2.012195	1.846342	-1.71801	-6.91413	-7.93763	8.780488	15.97561	1.768293	20.47761
6	0.003176	-10.9102	-5.95164	-7.3459	-3.77055	-1.82927	0.487805	-2.98781	-1.34279	-7.15994	-6.67618	1.371218	-3.65854	-11.2195	2.012195	1.846342	-1.71721	-6.91528	-7.93734	8.780488	15.97561	1.768293	14.26719
7	0.003971	-12.253	-5.95046	-7.34673	-3.77134	-1.82927	0.487805	-2.98781	-1.84634	-7.15938	-6.6758	1.371977	-3.65854	-11.2195	2.012195	1.510643	-1.71638	-6.91644	-7.93706	8.780488	15.97561	1.768293	8.392464
8	0.004765	-11.4138	-5.94925	-7.34756	-3.77215	-1.82927	0.487805	-2.98781	-2.18204	-7.1588	-6.67544	1.372698	-3.65854	-11.2195	2.012195	-1.67849	-1.71551	-6.91761	-7.93678	8.780488	15.97561	1.768293	19.97406
9	0.005559	-11.9173	-5.94801	-7.34841	-3.77298	-1.82927	0.487805	-2.98781	-0.50355	-7.15821	-6.67512	1.373378	-3.23171	-11.1585	1.829268	-0.16785	-1.7146	-6.9188	-7.9365	8.963415	15.91464	1.646341	17.12062
10	0.006353	-11.2459	-5.94672	-7.34927	-3.77383	-1.64634	0.426829	-2.98781	-0.83925	-7.15759	-6.67483	1.374014	-3.23171	-11.1585	1.829268	1.846342	-1.71365	-6.91999	-7.93623	8.963415	15.91464	1.646341	10.5745
11	0.007147	-14.2672	-5.9454	-7.35014	-3.77469	-1.64634	0.426829	-2.98781	0.503548	-7.15696	-6.67457	1.374605	-3.23171	-11.1585	1.829268	2.853437	-1.71266	-6.92119	-7.93596	8.963415	15.91464	1.646341	6.71397
12	0.007941	-13.0922	-5.94405	-7.35103	-3.77558	-1.64634	0.426829	-2.98781	1.678492	-7.1563	-6.67434	1.375148	-3.23171	-11.1585	1.829268	0.335699	-1.71163	-6.9224	-7.93569	8.963415	15.91464	1.646341	6.546121
13	0.008735	-15.2743	-5.94286	-7.35193	-3.77648	-1.64634	0.426829	-2.98781	-0.50355	-7.15563	-6.67414	1.375641	-3.23171	-11.1585	1.829268	-1.51064	-1.71055	-6.92362	-7.93542	8.963415	15.91464	1.646341	5.706875
14	0.009529	-14.2672	-5.94123	-7.35284	-3.77741	-1.64634	0.426829	-2.98781	-0.6714	-7.15494	-6.67398	1.376081	-3.23171	-11.1585	1.829268	1.678492	-1.70944	-6.92484	-7.93515	8.963415	15.91464	1.646341	-0.16785
15	0.010324	-13.5958	-5.93976	-7.35376	-3.77835	-1.64634	0.426829	-2.98781	1.174945	-7.15422	-6.67386	1.376467	-3.23171	-11.1585	1.829268	2.18204	-1.70828	-6.92608	-7.93488	8.963415	15.91464	1.646341	-2.85344
16	0.011118	-17.6242	-5.93825	-7.3547	-3.7793	-1.64634	0.426829	-2.98781	1.846342	-7.15349	-6.67378	1.376795	-3.23171	-11.1585	1.829268	-0.16785	-1.70708	-6.92732	-7.93461	8.963415	15.91464	1.646341	-7.21752
17	0.011912	-17.2885	-5.93671	-7.35564	-3.78028	-1.64634	0.487805	-3.04878	-0.3357	-7.15274	-6.67373	1.377065	-3.23171	-10.9146	1.829268	-1.84634	-1.70584	-6.92857	-7.93434	8.780488	15.42683	1.890244	2.853437
18	0.012706	-13.9315	-5.93561	-7.3566	-3.78127	-1.64634	0.487805	-3.04878	-2.18204	-7.15197	-6.67372	1.377273	-3.23171	-10.9146	1.829268	-2.51774	-1.70456	-6.92983	-7.93407	8.780488	15.42683	1.890244	5.874724

Figure 1- EMG signal data



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	
1	Time	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	RightHand	
2	0	80.85325	-127.456	252.6471	0.598039	-0.20365	0.330144	0.701342	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10275	-0.16653	0.969724	33.42196	3.
3	0.003333	80.85487	-127.448	252.6499	0.598085	-0.20374	0.33016	0.701269	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10275	-0.16653	0.969725	33.42196	3.
4	0.006667	80.85127	-127.453	252.6494	0.598042	-0.20359	0.330259	0.701304	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10274	-0.16652	0.969727	33.42196	3.
5	0.01	80.85488	-127.435	252.6345	0.598066	-0.2039	0.330177	0.70123	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10274	-0.16652	0.969727	33.42196	3.
6	0.013333	80.84582	-127.428	252.6373	0.59827	-0.20392	0.330172	0.701054	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10272	-0.16651	0.96973	33.42196	3.
7	0.016667	80.84826	-127.429	252.64	0.59809	-0.20385	0.330253	0.701189	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.1461	-0.10272	-0.16651	0.96973	33.42196	3.
8	0.02	80.84196	-127.412	252.6358	0.59836	-0.20408	0.330235	0.700902	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14611	-0.10271	-0.1665	0.969733	33.42196	3.
9	0.023333	80.82876	-127.396	252.6289	0.59828	-0.20422	0.330156	0.700966	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14611	-0.10271	-0.1665	0.969733	33.42196	3.
10	0.026667	80.86326	-127.396	252.6283	0.598242	-0.2043	0.330267	0.700921	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14611	-0.10269	-0.16649	0.969735	33.42196	3.
11	0.03	80.84009	-127.393	252.6366	0.598247	-0.20431	0.33023	0.700933	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14611	-0.10269	-0.16649	0.969736	33.42196	3.
12	0.033333	80.833	-127.382	252.6407	0.598568	-0.20444	0.330189	0.70064	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14612	-0.10267	-0.16648	0.969739	33.42196	3.
13	0.036667	80.84636	-127.367	252.6291	0.598312	-0.20471	0.330111	0.700815	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14612	-0.10267	-0.16648	0.969739	33.42196	3.
14	0.04	80.8444	-127.373	252.6389	0.598228	-0.20466	0.330199	0.700863	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14612	-0.10265	-0.16647	0.969742	33.42196	3.
15	0.043333	80.83883	-127.348	252.6375	0.59832	-0.20481	0.33016	0.700757	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14612	-0.10265	-0.16647	0.969742	33.42196	3.
16	0.046667	80.82932	-127.365	252.6511	0.598359	-0.20475	0.33019	0.700727	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14613	-0.10264	-0.16646	0.969745	33.42196	3.
17	0.05	80.8322	-127.365	252.6542	0.598298	-0.20489	0.330195	0.700738	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14613	-0.10263	-0.16645	0.969746	33.42196	3.
18	0.053333	80.84426	-127.354	252.6639	0.59828	-0.20498	0.330299	0.700675	4.952467	14.23313	1.898188	-0.51245	-0.04695	0.286383	0.808192	22.53108	0	0	-0.14613	-0.10262	-0.16644	0.969749	33.42196	3.

Figure 2- Processed data

Note: The python script used in this section can be found in the appendix.

Library Imports

```
import json
```

```
import pandas as pd
```

```
import numpy as np
```

These libraries are imported to support data handling and processing.

-The json module is used to load and parse the raw JSON recording file.

-numpy is used for efficient numerical operations and array manipulation.

-pandas provides DataFrame structures that simplify organizing, labeling, and exporting the data to CSV format.

Step 1: Load the JSON File

with open('Dynamic_2.json', 'r') as file:

```
data = json.load(file)
```

The JSON file containing the recorded EMG and movement data is opened and loaded into memory as a Python dictionary. This allows structured access to all sections of the dataset, including analog sensor readings and skeletal motion data.

Step 2: Extract EMG (Analog) Data

Locate the EMG Data Block

```
analog_section = next(item for item in data['Analog'] if item['Name'] == 'Delsys Trigno API')
```

The EMG data is stored within the *Analog* section of the JSON file. This line searches for and selects the entry labeled “Delsys Trigno API”, which contains the EMG and IMU sensor measurements recorded during the experiment.



Extract EMG Sampling Frequency

```
emg_frequency = analog_section['Timebase']['Frequency']
```

The EMG sampling frequency is extracted from the metadata. This value is critical for reconstructing the time axis and ensuring that the signal timing is accurate during analysis.

Extract Channel Labels

```
emg_labels = analog_section['Labels']
```

Sensor labels corresponding to each EMG and IMU channel are read directly from the file. Preserving these labels ensures traceability between recorded data and physical sensors.

Load and Reformat EMG Values

```
emg_values = np.array(analog_section['Values']).T
```

The recorded EMG values are converted into a NumPy array and transposed so that each row represents a single time sample and each column corresponds to one sensor channel. This orientation is required for proper DataFrame construction.

Reconstruct EMG Time Axis

```
emg_time = np.arange(emg_values.shape[0]) / emg_frequency
```

A time vector is reconstructed by dividing the sample indices by the known sampling frequency. This creates a precise timestamp for each EMG sample.

Create EMG DataFrame

```
emg_df = pd.DataFrame(emg_values, columns=emg_labels)
```

```
emg_df.insert(0, 'Time', emg_time)
```

The EMG data and channel labels are organized into a Pandas DataFrame. The reconstructed time axis is inserted as the first column to facilitate synchronization with other data sources and downstream analysis.

Step 3: Extract Movement (Skeleton) Data

Access Skeleton Data and Sampling Frequency

```
skeleton_data = data['Skeletons'][0]
```

```
skeleton_frequency = skeleton_data['Timebase']['Frequency']
```

The movement data is retrieved from the *Skeletons* section of the JSON file. Since the recording involves a single subject, the first skeleton entry is used. The sampling frequency is extracted to reconstruct the movement time axis.



Retrieve Body Segments

```
segments = skeleton_data.get('Segments', [])
```

All tracked body segments (such as hand and finger segments) are accessed. Each segment contains position and orientation data recorded over time.

Initialize Storage for Segment Data

```
movement_dfs = {}
```

A dictionary is initialized to store individual DataFrames for each body segment before merging them into a single dataset.

Process Each Body Segment

for segment in segments:

```
    segment_name = segment['Name']
```

Each body segment is processed individually, allowing separate handling of position and orientation data while preserving segment identity.

Extract Position and Orientation Values

```
values = segment['Parts'][0]['Values']
```

```
values_array = np.array(values)
```

For each segment, three-dimensional position data and quaternion-based orientation data are extracted and converted into a numerical array for processing.

Reconstruct Movement Time Axis

```
time_axis = np.arange(values_array.shape[0]) / skeleton_frequency
```

A time vector is reconstructed for the movement data using the skeleton sampling frequency, ensuring accurate temporal alignment.

Create Segment DataFrame

```
df = pd.DataFrame(values_array, columns=[
```

```
    f'{segment_name}_X',
```

```
    f'{segment_name}_Y',
```

```
    f'{segment_name}_Z',
```

```
    f'{segment_name}_Qx',
```




```
f {segment_name}_Qy',
f {segment_name}_Qz',
f {segment_name}_Qw',
])
df.insert(0, 'Time', time_axis)
movement_dfs[segment_name] = df
```

A DataFrame is created for each segment, with clearly labeled columns representing position and orientation. The time axis is added as the first column, and the DataFrame is stored for later merging.

Merge All Segment Data

```
movement_df = pd.concat(
    [df.set_index('Time') for df in movement_dfs.values()],
    axis=1
).reset_index()
```

All segment DataFrames are merged into a single DataFrame using time as a common reference. This aligns all movement data temporally and produces a consolidated dataset containing all tracked body segments.

Step 4: Export Data to CSV Files

```
emg_df.to_csv('emg_data.csv', index=False)
movement_df.to_csv('movement_data.csv', index=False)
```

The processed EMG data and the consolidated movement data are saved as separate CSV files. This format allows straightforward integration with signal processing, visualization, and simulation tools used in later stages of the project.

Completion Confirmation

```
print(" ■ EMG and movement data successfully extracted and saved as CSV files.")
```

A confirmation message is printed to indicate successful execution of the extraction and conversion process.



3. Signals Preprocessing

To ensure that the EMG signals collected for the index- and pinky-finger gestures were suitable for accurate analysis and channel selection, a complete preprocessing pipeline was applied. Each step addresses a specific limitation of raw EMG data and prepares the signal for reliable feature extraction and interpretation.

We'll go through the different steps for processing the **EMG signal of channel 12 of the index finger** as an example

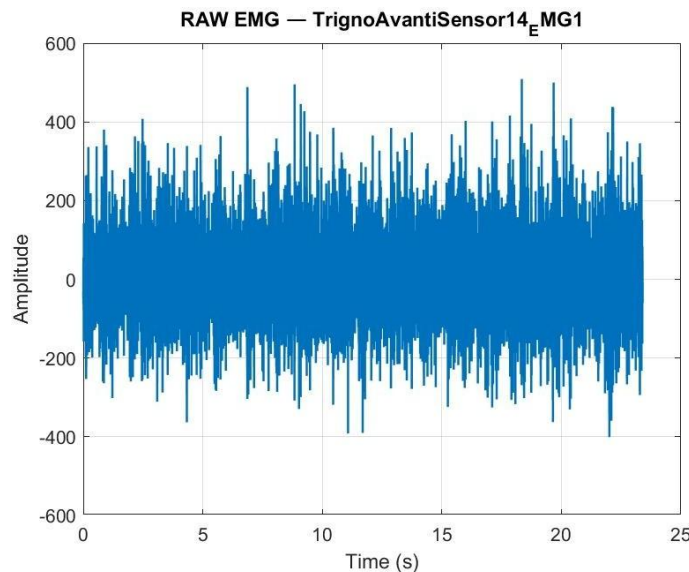


Figure 3- Raw EMG signal of channel 14 of pinky

3.1. Bandpass Filtering (10–500 Hz)

Surface EMG signals have both very low-frequency noise, like movement and electrode drift, and high-frequency disturbances, such as electrical interference and sensor noise. A 4th-order Butterworth bandpass filter with cutoffs at 10-500 Hz was used to isolate the true physiological content of the EMG. The low-cut frequency eliminates baseline drift and motion artifacts. The high-cut frequency removes high-frequency noise that does not provide useful muscle information. Bandpass filtering ensures that only the electrical activity generated by muscles is retained, yielding a clean signal for further analysis.

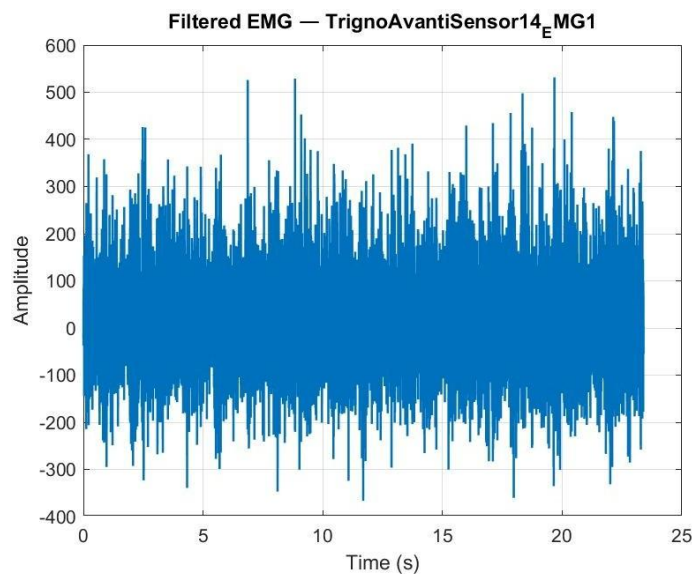


Figure 4- Filtered EMG signal for channel 14 of pinky

3.2. Full-Wave Rectification

The filtered EMG waveform is bipolar because muscle action potentials naturally fluctuate above and below zero. Full-wave rectification changes the entire signal to positive values by taking the absolute value of each sample. This step is necessary because many EMG metrics, such as RMS, MAV, and envelope extraction, focus on the signal's magnitude instead of its direction. Rectification changes the oscillating EMG waveform into a form that directly shows the intensity of muscle activation.

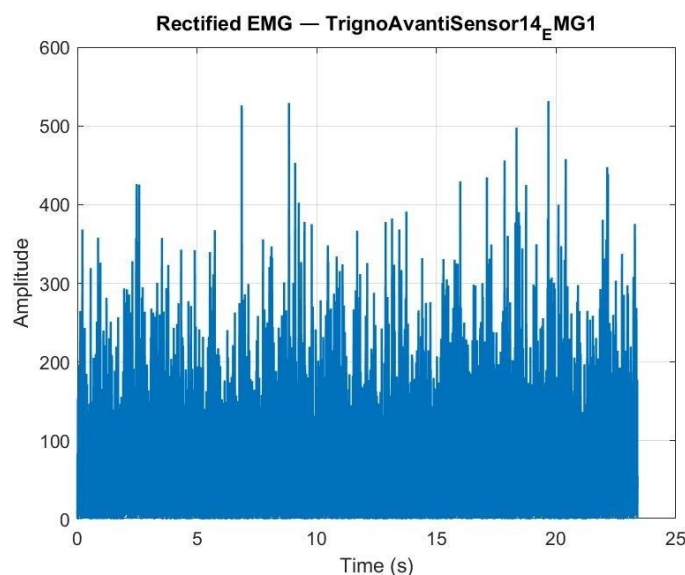


Figure 5- Rectified EMG signal for channel 14 of pinky

3.3. Normalization (Scaling to Maximum Value)

Raw EMG amplitudes can differ significantly between channels due to factors like electrode placement, skin conductivity, and sensor sensitivity. To make channels comparable, each rectified EMG signal was normalized by dividing it by its own maximum value. Normalization adjusts all channels to a 0-1 range, allowing fair comparison among different sensors and preventing amplitude-based features from being



influenced by raw sensor gain. This step is vital for consistently identifying dominant EMG channels across various gestures.

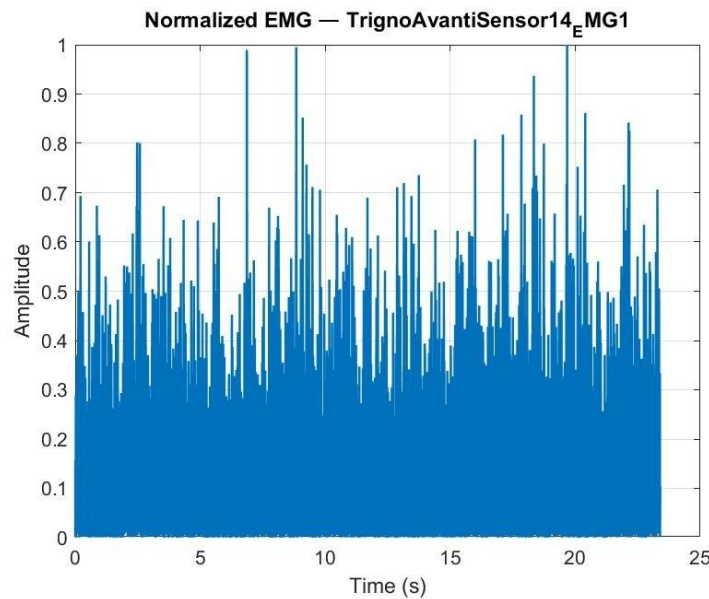


Figure 6- Normalized EMG signal for channel 14 of pinky

4. Time-domain Analyses

Time-domain analysis offers measurable details about the EMG waveform by looking at how the signal acts over time. For the two selected gestures, index-finger contraction/relaxation and pinky-finger contraction/relaxation, these metrics help identify which muscles, and therefore which EMG channels, show stronger or more consistent activation. The following five metrics were calculated for all EMG channels to describe muscle activity during each gesture.

4.1. Mean Absolute Value (MAV)

It is defined by computing average of the absolute EMG values within a time segment. MAV shows the overall level of muscle activity by measuring the average signal strength. A channel with a higher MAV indicates a muscle that remains more active throughout the gesture.

During index or pinky contractions, the muscles for each finger should display noticeably different MAV values. By comparing MAV across channels, we can easily determine which sensors capture the strongest and most sustained activation for each gesture. This information helps in selecting the dominant channel.

Let:

- $x[n]$: be the EMG signal,
- N : be the number of samples in the analysis window.

$$MAV = \frac{1}{N} \sum_{n=1}^N |x[n]|$$



4.2. Root Mean Square (RMS)

It is defined by the square root of the mean of squared EMG values. RMS reflects the effective power of the EMG signal. Since squaring emphasizes larger values, RMS highlights periods of strong, burst-like muscle activations and provides a stable measure of contraction intensity.

RMS is one of the most dependable indicators of muscle engagement during finger contractions. Channels with the highest RMS values during a gesture correspond to muscles generating the greatest contraction intensity. This directly aids in identifying which EMG channels are most involved in index or pinky finger movements.

$$RMS = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2}$$

4.3. Waveform Length (WL)

It is defined by the cumulative sum of absolute differences between consecutive samples. WL measures the complexity and variability of the EMG signal. A high WL indicates rapid signal changes, which usually correspond to fast or dynamic muscle activation with many shifts in amplitude.

Different finger contractions activate distinct groups of motor units. The muscles for the index finger might produce more dynamic firing patterns compared to the pinky, or vice versa. These differences would show up as variations in waveform length. WL helps identify channels with stable contractions versus those with more complex muscle activity.

$$WL = \sum_{n=1}^N |x[n+1] - x[n]|$$

4.4. Integrated EMG (iEMG)

It is defined by the sum of the absolute EMG values over a defined time interval. iEMG measures the total amount of muscle effort or workload during the gesture. Unlike MAV or RMS, which describe instantaneous intensity, iEMG represents the total accumulated activity during the contraction.

Because index and pinky contractions may involve muscles that differ in intensity, duration, and total effort, iEMG helps identify the channel that represents the most overall muscle work. This is especially helpful when contractions vary or when comparing long versus short activation patterns.

$$iEMG = \sum_{n=1}^N |x[n]|$$

4.5. Slope Sign Changes (SSC)

It is defined by the number of times the slope (first derivative) of the EMG signal changes sign, ignoring small fluctuations below a threshold. SSC relates to the frequency content of the EMG signal. More slope sign changes indicate quicker firing activity or sudden shifts in contraction force.



When contracting the index or pinky finger, different muscle groups may show various firing behaviors. A channel with a high SSC suggests quick or irregular bursts of muscle activity, often seen in finer or more unstable contractions. SSC helps identify which channels reflect more complex muscular engagement during each gesture.

$$SSC = \sum_{n=2}^{N-1} f((x[n] - x[n-1]) \cdot (x[n] - x[n+1]))$$

Where the decision function is:

$$f(y) = \begin{cases} 1, & \text{if } y > \epsilon \\ 0, & \text{otherwise} \end{cases}$$

ϵ : is a small threshold to ignore noise

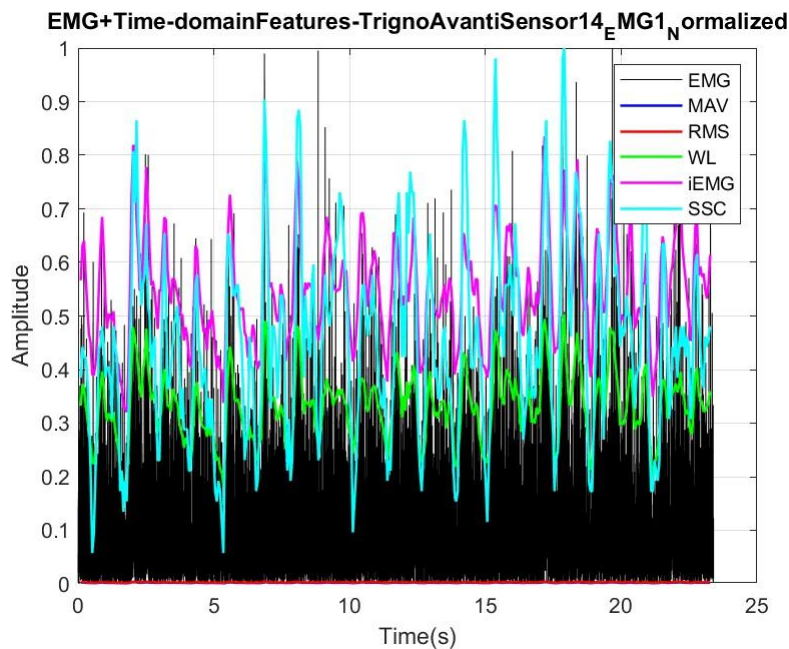


Figure 7- Time-domain analyses for processed EMG signal channel 14 pinky

4.6. Hurst Component (Hurst Exponent)

The Hurst component, commonly expressed as the Hurst exponent (H), is a statistical measure used to quantify the long-term temporal dependence and self-similarity of a signal. It characterizes whether a time series exhibits persistent behavior, random behavior, or anti-persistent behavior. Values of the Hurst exponent range between 0 and 1. A value of $H \approx 0.5$ indicates a purely random process with no long-term correlation, $H > 0.5$ denotes persistent behavior where increases in the signal are likely to be followed by further increases, and $H < 0.5$ reflects anti-persistent behavior with frequent reversals.

When applied to EMG signals, the Hurst exponent provides insight into the underlying muscle activation dynamics. Persistent behavior is typically associated with sustained or coordinated muscle recruitment, while lower values indicate more irregular or rapidly fluctuating activity. The formula used can be found in appendix.



Why RMS and iEMG Are the Best Metrics for Dominant Channel Selection

RMS is the Best Indicator of Instantaneous Muscle Intensity as it measures contraction strength by reflecting the effective power of the EMG signal. In this project, the channel with the highest RMS corresponds to the muscle producing the strongest contraction during the index or pinky gesture. This makes RMS ideal for identifying primary activation channels.

iEMG is the Best Indicator of Total Muscle Effort as it captures the full accumulated activity during the contraction period. In this project, the channel with the highest iEMG represents the muscle responsible for the largest overall workload, even if the contraction intensity fluctuates.

Together, RMS and iEMG provide a complete understanding of muscle dominance:

- a) RMS answers the question, "How strong was the muscle at any given moment?"
- b) iEMG answers the question, "How much total effort did the muscle contribute overall?"

This combination ensures that dominant channels are chosen wisely for both quick bursts (RMS) and sustained contractions (iEMG). This is essential for later control and gesture recognition of the prosthetic hand.

5. Frequency-domain Analyses

When we analyze EMG signals, time-domain features such as RMS, MAV, and iEMG indicate the strength and activity of the muscle. EMG also holds vital information about the frequency content of muscle activity; this is where frequency-domain analysis plays a role. **As a demonstration, also, channel 14 of the pinky finger is selected.**

To extract this information, we follow two main steps:

5.1. Converting EMG to the Frequency Domain (FFT)

Fast Fourier Transform (FFT) is applied to each EMG window. The reason for using FFT is that it converts the EMG signal from the time domain to the frequency domain. It reveals:

- a) What frequencies are present in the muscle signal
- b) How strong each frequency component is

EMG frequency components relate to how the FFT breaks down a signal into its individual sinusoidal parts, showing which frequencies are present and their relative strengths. It offers a glimpse of the signal's spectrum.

Its units are usually in amplitude (e.g., volts) and are sensitive to signal length and windowing. The FFT is great for spotting dominant frequencies or checking the bandwidth of a signal.

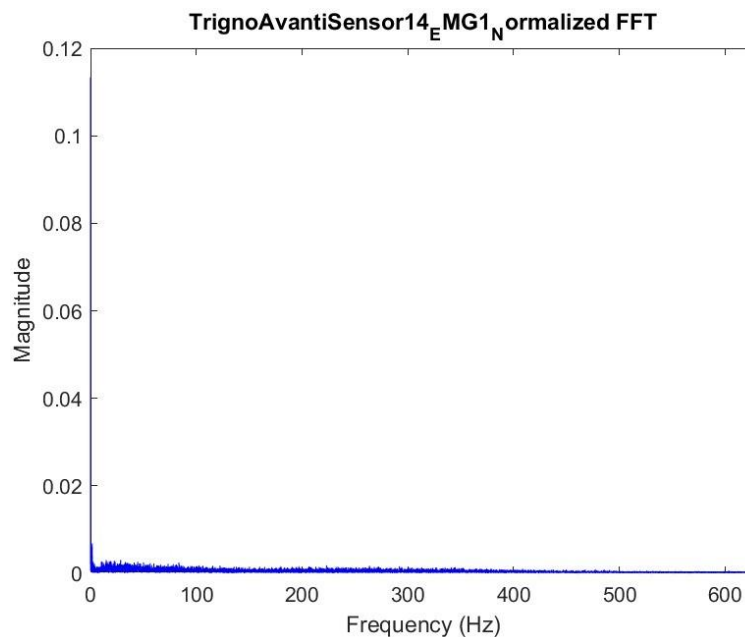


Figure 8- FFT for channel 14 of pinky

Note: The FFT graph may look empty.

The figure shows the normalized frequency spectrum of the preprocessed EMG signal obtained using the Fast Fourier Transform (FFT). The FFT reveals that most of the signal energy is concentrated at low frequencies, with magnitude decreasing steadily as frequency increases. This is expected due to the rectification and envelope-related components of the EMG signal, which emphasize overall muscle activation rather than individual motor unit spikes. The absence of prominent high-frequency peaks indicates effective noise suppression during preprocessing.

5.2. Power Spectral Density (PSD)

After the FFT, we calculate the Power Spectral Density (PSD), usually using Welch's method as the PSD shows how power is spread across frequency bands. Instead of just raw frequency amplitudes, it indicates how much energy the muscle generates at each frequency. The PSD illustrates how the signal's power is divided based on frequency, typically in units of dB/Hz. It measures energy, not raw amplitude.

The PSD uses averaging (like the Welch method) to lower variance, resulting in a smoother, clearer estimate. It enables comparison between signals of different lengths.

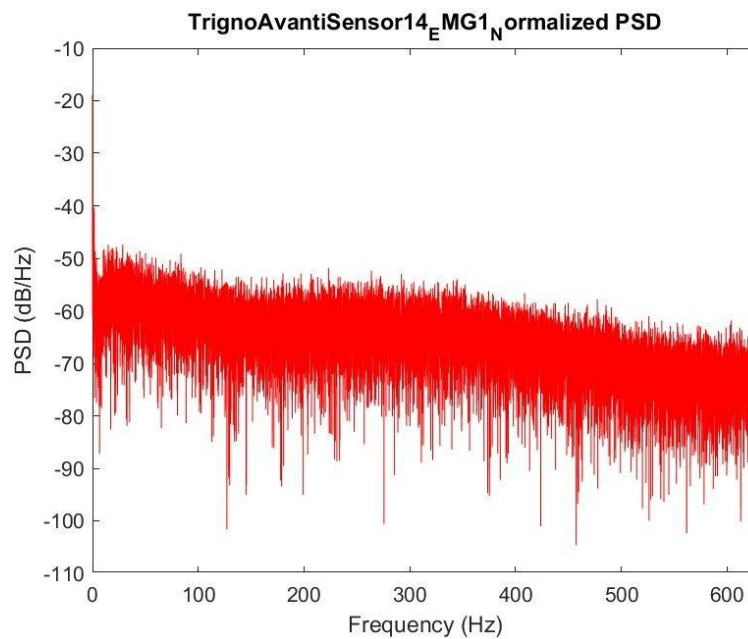


Figure 9- PSD for channel 14 of pinky

The PSD illustrates how the signal power is distributed across frequency. Most of the energy is concentrated in the low-to-mid frequency range, which is consistent with the typical spectral content of surface EMG signals after band-pass filtering. As frequency increases, the power gradually decreases, indicating reduced contribution of higher-frequency components, mainly due to physiological muscle behavior and the applied filtering stages. The absence of sharp dominant peaks suggests that the signal is broadband in nature, as expected for EMG activity, rather than being driven by a single periodic component.

6. Time-frequency Analyses

FFT and PSD analyze the signal in the frequency domain only. They require that the signal's properties remain constant over time. However, EMG is very non-stationary, with bursts of activity happening at specific times. The Discrete Wavelet Transform (DWT) solves this by providing time-frequency localization.

6.1. Discrete Wavelet Transform (DWT)

DWT breaks down the signal into:

- a) Detail coefficients (D1, D2, D3, D4, D5) → high-frequency components
- b) Approximation coefficients (A5) → low-frequency trends

Each level of decomposition corresponds to a specific frequency band.

Interpretation of the Provided DWT Plots

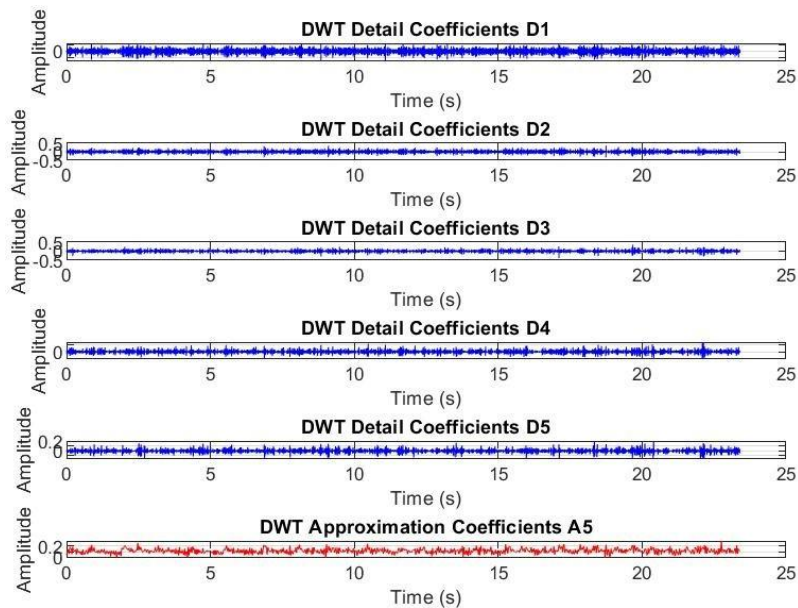


Figure 10- DWT plots for channel 14 of pinky

The plot shows five layers of detail coefficients (D1–D5) and one approximation level (A5):

The detail coefficients (D1 to D5) capture the high- to mid-frequency components of the EMG signal. The higher-order details (D1 and D2) mainly represent rapid, low-amplitude fluctuations associated with high-frequency muscle activity and residual noise. As the decomposition level increases (D3 to D5), the coefficients reflect progressively lower-frequency content with slightly higher amplitude variations, indicating more sustained muscle activation patterns. The relatively stable amplitude across time suggests consistent muscle activity without abrupt transient events.

The approximation coefficients (A5) represent the low-frequency component of the signal and describe the overall signal trend. This component shows smoother variations and contains the dominant energy of the EMG after preprocessing, reflecting the envelope of muscle activation over time.

DWT offers:

- Temporal localization: when muscle activation bursts happen
- Frequency localization: which frequency bands dominate during each event
- Noise separation: high-frequency details mostly consist of noise after RMS filtering
- Feature extraction capability: useful for classification or onset detection

Therefore, DWT is important for analyzing EMG when muscle contractions vary over time.



7. Dominant Channel Selection

To find the EMG channels that contribute most to each gesture, we conducted a dominant-channel selection procedure using time-domain feature analysis. The two gestures examined in this project, index-finger contraction and pinky-finger contraction, activate different muscle groups in the forearm. Therefore, selecting the channels that show the highest and most consistent activation is crucial for accurate gesture recognition and reliable control of the prosthetic hand in later stages.

We used two main time-domain metrics for this purpose:

- (1) RMS (Root Mean Square) and
- (2) iEMG (Integrated EMG).

These features were selected because they directly reflect muscle activation, intensity and total muscle effort. RMS measures the effective power of the EMG signal, making it sensitive to strong contractions. Meanwhile, iEMG represents the accumulated activity throughout the gesture, capturing sustained or extended activation that may not show up in short-duration peaks.

For the next stage, we performed two procedures:

- a) Weighted average for the two dominant channels of each gesture as follows: **Output EMG signal = 0.55 * the most dominant channel + 0.45 * the second most dominant one**; giving the higher priority for the channel showing the strongest activation.
- b) RMS sliding window onto the resulting signal. Even after rectification, the EMG signal still shows rapid, high-frequency fluctuations from individual motor unit firings. These quick changes make it hard to see muscle activation levels directly. A sliding-window Root Mean Square (RMS) envelope was calculated to create a smooth and meaningful representation of muscle activity. RMS averaging acts like a low-pass filter, dampening high-frequency spikes and highlighting the slower changes linked to muscle force. The resulting RMS envelope shows the overall muscle activation pattern over time, making it suitable for:
 - Comparing channel activation across gestures.
 - Identifying dominant muscle channels,
 - Extracting reliable time-domain features
 - Preparing the signal for classification or control.



6.1. First gesture: Index Contraction/ Relaxation

We computed RMS and iEMG values across all channels during the contraction interval. The channels that displayed

the highest RMS and highest iEMG were identified as the dominant contributors for each gesture. According to the analysis table, Channel 8 showed the greatest muscle activation for one gesture, with the highest RMS (50.6) and iEMG (8,824). For the second gesture, Channel 5 consistently demonstrated the next strongest activation levels (RMS = 44.98, iEMG = 8,168). These results indicate

that Channels 8 and 5 capture the most relevant muscle activity associated with contraction of the index finger. Therefore, we selected these channels as the dominant EMG channels for later control and classification tasks.

Channels	iEMG	RMS
1	6443	38.98
2	6839	38.96
3	5362	32.08
4	5610	29.35
5	8168	44.98
6	5103	29.58
7	2981	15.98
8	8824	50.6
9	5778	32.25
10	3479	20.9
11	6355	32.52
12	5830	33.42
13	5629	29.35
14	5022	28.29

Justification for Using Time-Domain Features Instead of Frequency-Domain or Time-Frequency Methods

a) Muscle activation strength is best shown through amplitude-based features.

Dominance in EMG channels mainly depends on how strongly the underlying muscle contracts, which directly impacts signal amplitude. Measures like RMS and iEMG provide a clear and physiologically relevant representation of muscle effort. In contrast, frequency-domain features (e.g., MNF, MDF) describe spectral shape or fatigue characteristics, which do not reliably indicate activation strength.

b) Finger-gesture tasks show clear amplitude differences but minimal spectral variation.

Index and pinky finger contractions mainly differ in activation intensity, not in changes in frequency content. The spectral characteristics between channels were similar and therefore not useful for determining dominance. Since this stage focuses on identifying which muscle fires the strongest, amplitude-based metrics are more suitable.



- c) Time-frequency analysis is suited for dynamic or rapidly changing tasks.

Methods like STFT or wavelet transforms work well when EMG patterns change quickly over time or include transient events. The gestures in this project involve relatively simple contraction and relaxation cycles, where time-frequency decomposition does not provide any added advantage over direct amplitude-based evaluation.

- d) Time-domain features are recommended by EMG standards for channel selection.

In the literature on prosthetics and biomechanics, RMS and iEMG are standard metrics for measuring muscle engagement and selecting representative channels. These metrics correlate strongly with muscle force generation, making them ideal for identifying channels suitable for later control algorithms.

6.1.1. Conclusion of Channel Selection for index

Based on RMS and iEMG values, Channels 5 and 8 were identified as the most dominant EMG channels for the chosen gesture. Using time-domain features provided a clear and direct basis for determining which muscles contributed most to each finger movement. These dominant channels will serve as the primary control inputs in the next simulation and control stages of the project.

Here's a snippet of the weighted average being calculated on the two dominant channels:

Time	TrignoAvantiSensor8 EMG_Normalized	TrignoAvantiSensor5 EMG_Normalized	Dominant Channels Weighted Average
0	0.049750527	0.00599542	0.030060729
0.000794	0.130611561	0.021706828	0.081604431
0.001588	0.108412851	0.02640505	0.071509341
0.002382	0.05644665	0.015078741	0.037831091
0.003176	0.06432204	0.020911784	0.044787425
0.003971	0.047176113	0.028832688	0.038921572
0.004765	0.029313581	0.004760567	0.018264725
0.005559	0.022723695	0.015167434	0.019323378
0.006353	0.016133073	0.013505709	0.014950759
0.007147	0.048561678	0.006981663	0.029850671
0.007941	0.0692078	3.40E-05	0.03807959
0.008735	0.02859906	0.001697294	0.016493265
0.009529	0.013682778	0.000264517	0.007644561
0.010324	0.009338648	0.025028517	0.016399089
0.011118	0.00550604	0.080232069	0.039132753
0.011912	0.030991415	0.11081716	0.066913

After that, we conduct RMS envelope analysis onto the resulting signal in order to have the final signal that will be considered as an input to our controller inside our MATLAB model.

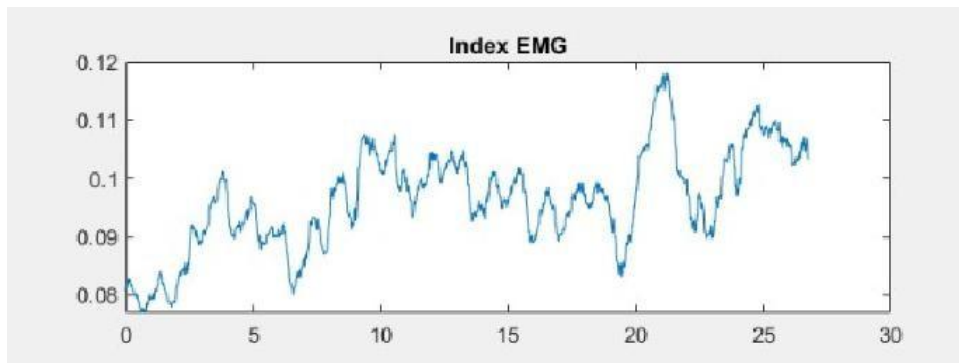


Figure 11-Weighted RMS EMG signal of index

Time	Dominant_Weighted_RMS
0	0.08070952
0.000794	0.080694883
0.001588	0.080756308
0.002382	0.080879783
0.003176	0.080952994
0.003971	0.080910547
0.004765	0.080974283
0.005559	0.080963098
0.006353	0.080923225
0.007147	0.080967736
0.007941	0.081303729
0.008735	0.081360147
0.009529	0.081412776
0.010324	0.081395187
0.011118	0.081358732
0.011912	0.081333896
0.012706	0.081326149
0.013500	0.081346208
0.014294	0.081401266

6.1.1.1. Some metrics for the two dominant channels for the index

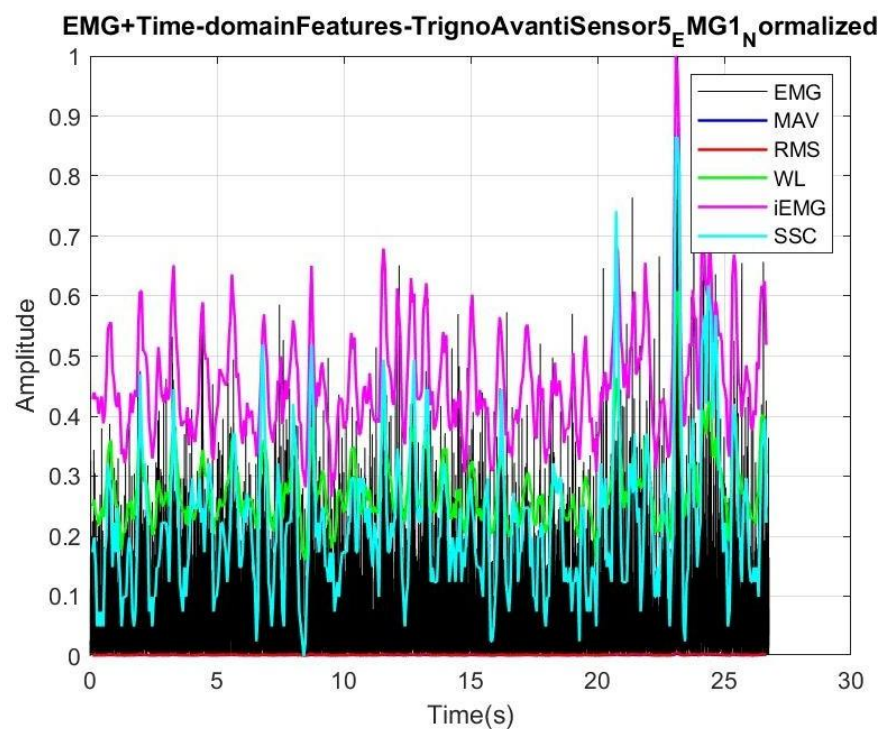


Figure 12- Time domain metrics for channel 5 of Index finger

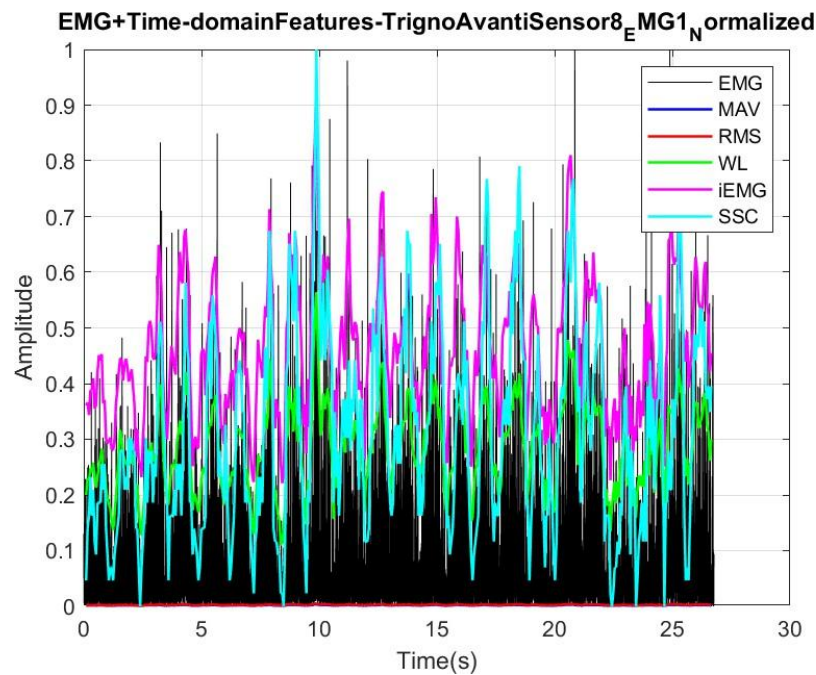


Figure 13- Time domain metrics for channel 8 of Index finger

A		B	
Sensor		Hurst	
TrignoAvantiSensor8_EMG1_Normalized		0.679530009	
TrignoAvantiSensor5_EMG1_Normalized		0.669601523	

Figure 14- Hurst Component for index dominant channels

For the index gesture, Channel 8 exhibits a Hurst exponent of **0.6795**, while Channel 5 shows a value of **0.6696**. Both values indicate a clear persistent structure, suggesting stable and sustained muscle activation during index finger movement. The slightly higher value for Channel 8 aligns with its identification as the most dominant chanl, reflecting stronger temporal consistency in its EMG activity.

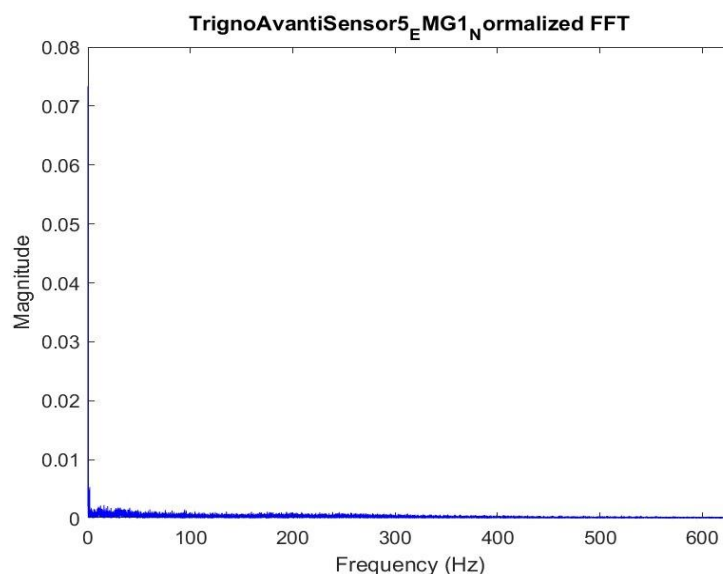


Figure 15- FFT for channel 5 of index finger

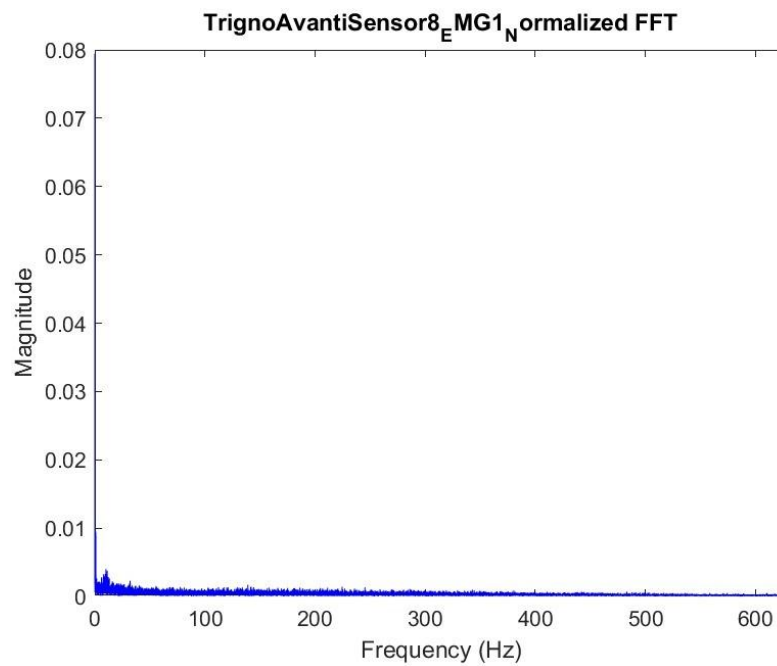


Figure 16- FFT for channel 8 of index finger

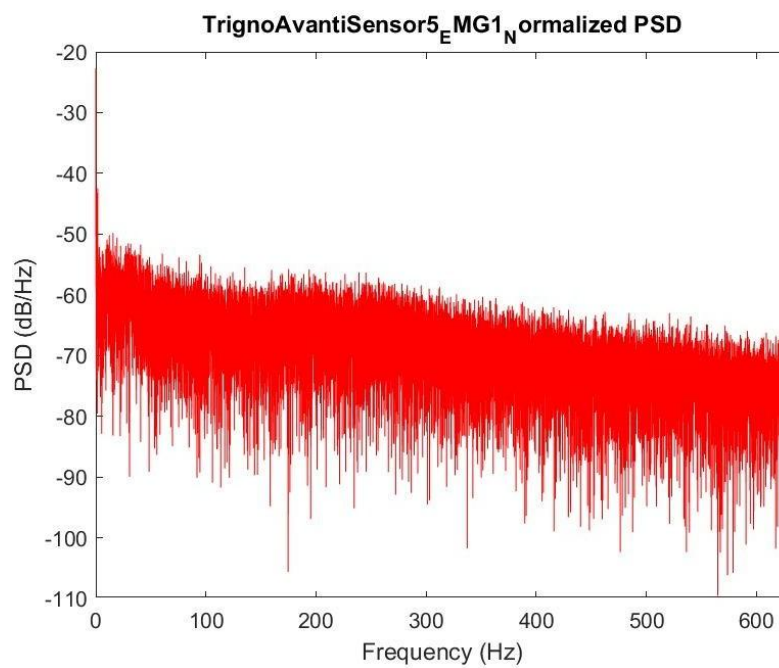


Figure 17- PSD for channel 5 of index finger

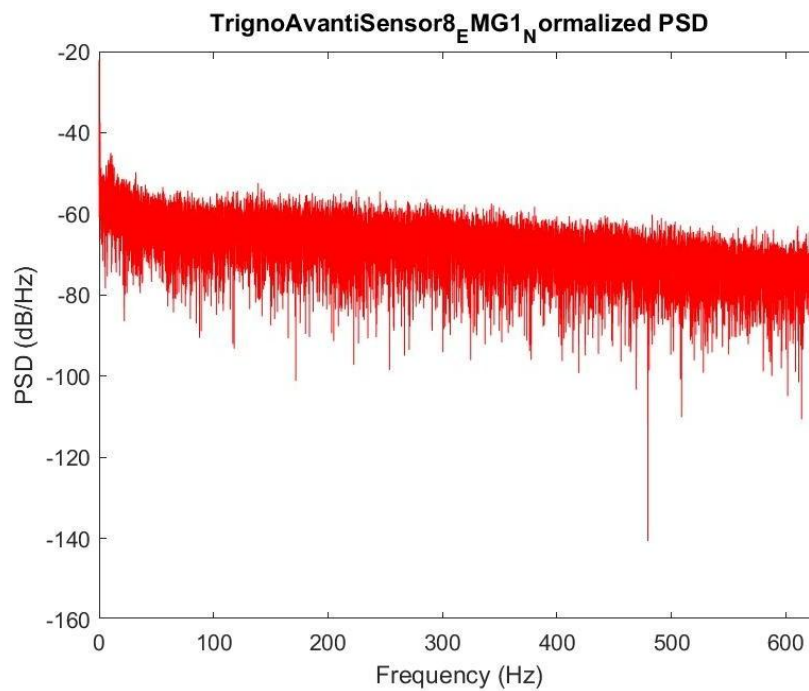


Figure 18- PSD for channel 8 of index finger

Dominant Channels Detected	PSD_BandPower_20_450Hz
TrignoAvantiSensor8_EMG1_Normalized	0.005223131
TrignoAvantiSensor5_EMG1_Normalized	0.004217099

Figure 19- PSD band power 20-450Hz for index finger

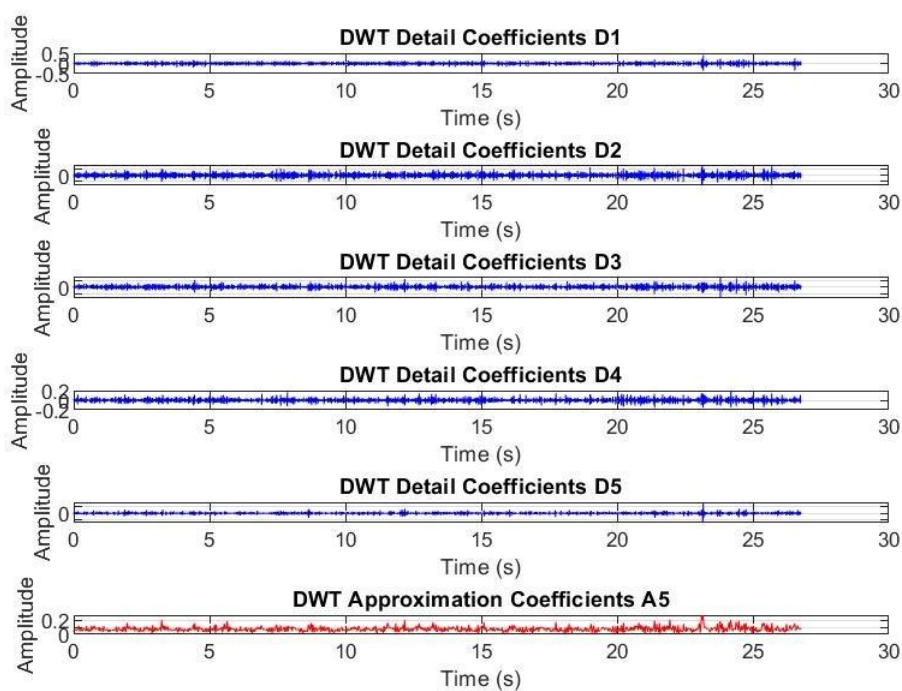


Figure 20- DWT for channel 5 of index finger

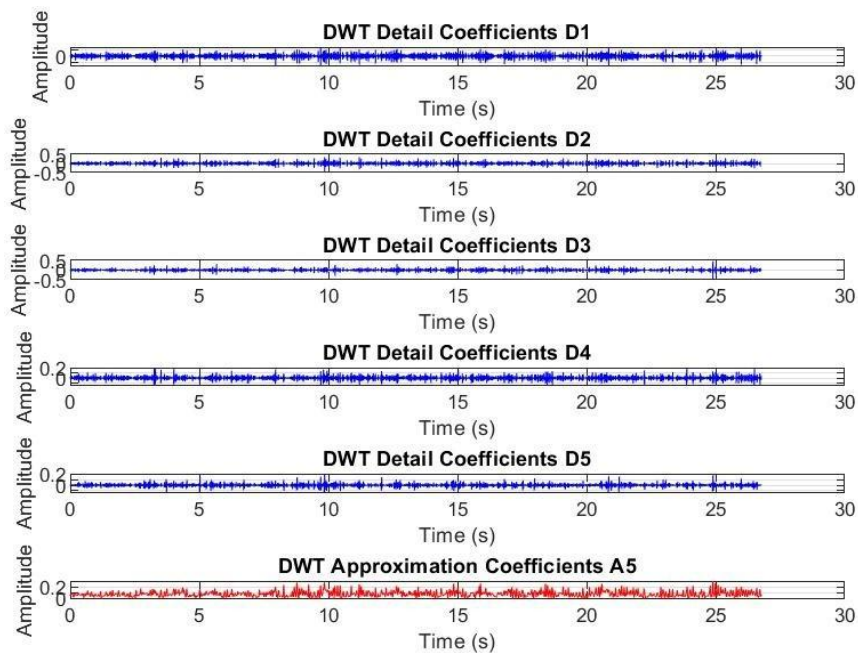


Figure 21- DWT for channel 8 of index finger

P	Q	R	S
TrignoAvantiSensor8_EMG1_Normalized_D3D4_Sum	TrignoAvantiSensor5_EMG1_Normalized_D3D4_Sum	Channels	Total summation of absolute D3,D4
-0.108601436	0.011651617	8	3660.9
-0.108601436	0.011651617	5	392.77
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		
-0.108601436	0.011651617		

Figure 22- Summation for absolute D3 & D4 for index dominant channels

The noticeably higher total summation of the absolute D3–D4 values for Channel 8 compared to Channel 5 indicates a stronger overall muscle activation or higher signal energy within the D3–D4 frequency band during the recording period.



6.2. Second gesture: Pinky Contraction/Relaxation

Here, we also computed the RMS and iEMG values across all channels during the contraction interval. The channels that displayed the highest RMS and highest iEMG were identified as the dominant contributors for this gesture. According to the analysis table, Channel 14 showed the greatest muscle activation for this gesture, with the highest RMS (59.53) and iEMG (15,001). Second to this channel, channel 6 consistently demonstrated the next strongest activation levels (RMS = 52, iEMG = 13,104).

Channel	iEMG	RMS
2	1989	7.915
1	8651	34.33
3	6322	25.117
6	13104	52
4	3169	12.59
7	12175	48.32
8	10091	10.053
10	7375	29.29
14	15001	59.53
11	7140	28.349
12	7014	27.848
9	2088	8.3
13	11162	44.299
5	11214	44.511

6.2.1. Conclusion of Channel Selection for pinky finger

These results indicate that Channels 6 and 14 capture the most relevant muscle activity associated with contraction of the pinky finger. Therefore, we selected these channels as the dominant EMG channels for later control and classification tasks.

Time	TrignoAvantiSensor6 EMG Normalized	TrignoAvantiSensor14 EMG Normalized	Dominant Channels Weighted Average
0	0.005295296	0.022504151	0.014760166
0.000794	0.024762659	0.05647873	0.042206498
0.001588	0.048146969	0.040912336	0.044167921
0.002382	0.045831907	0.018021085	0.030535955
0.003176	0.003917522	0.016412589	0.010789809
0.003971	0.071716973	0.002346763	0.033563357
0.004765	0.06109725	0.036567576	0.047605929
0.005559	0.020351555	0.069518846	0.047393565
0.006353	0.048399813	0.055015612	0.052038502
0.007147	0.04828099	0.157411941	0.108303013
0.007941	0.066942146	0.097065438	0.083509956
0.008735	0.030572725	0.042382982	0.037068367
0.009529	0.036205043	0.023097056	0.02899565
0.010324	0.046868517	0.025240219	0.034972953
0.011118	0.019254391	0.034017155	0.027373911
0.011912	0.014405883	0.004033362	0.008700997
0.012706	0.031375956	0.019818204	0.025019193
0.0135	0.066463364	0.005643747	0.033012574
0.014294	0.082104697	0.008449945	0.041594584

Here is a snippet of the excel sheet containing the finalized signal which is the result of the weighted average, but after performing sliding RMS window over the averaged signal, so that it is ready for being used inside the MATLAB model.

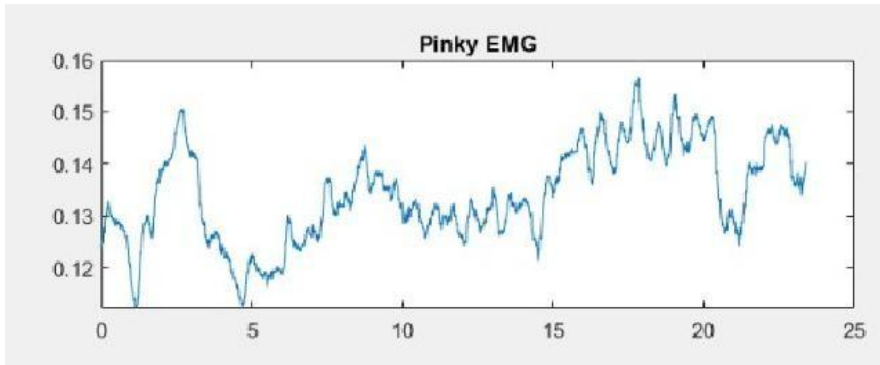


Figure 23- Weighted RMS EMG signal for pinky

Time	Dominant_Weighted_RMS
0	0.124438278
0.000794	0.124394288
0.001588	0.124335562
0.002382	0.124275924
0.003176	0.124232922
0.003971	0.124182293
0.004765	0.124122359
0.005559	0.124059192
0.006353	0.12399478
0.007147	0.1239322
0.007941	0.123884651
0.008735	0.123883635
0.009529	0.123859518
0.010324	0.123806117
0.011118	0.123784112
0.011912	0.123767219
0.012706	0.123739065

6.2.1.1. Some metrics for channel 6 for the pinky

Pinky has 2 dominant channels, 6 and 14, however throughout the report, we've taken 14 as an example. That's why now we're only focusing on the most dominant channel for this gesture, channel 6.

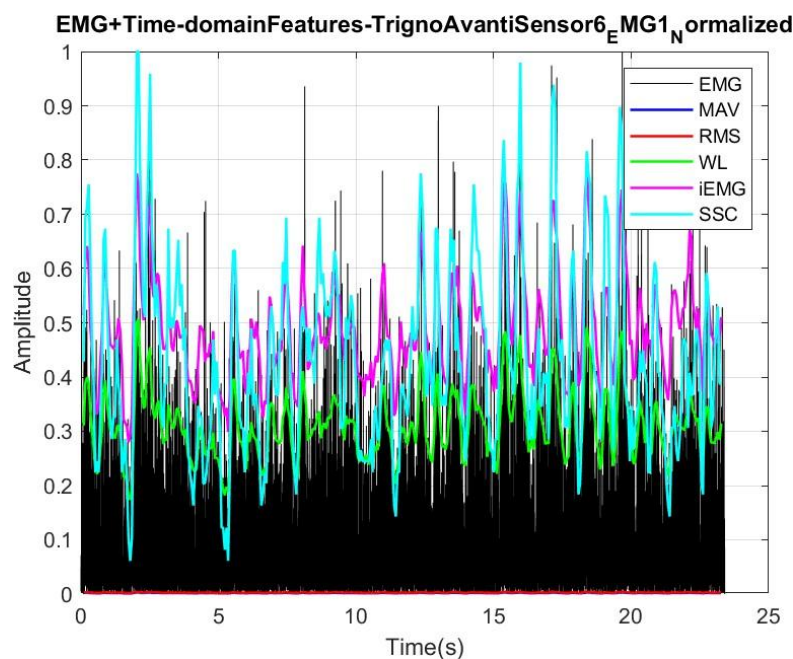


Figure 24- Time domain metrics for channel 6 of pinky finger

A		B
Sensor	Hurst	
TrignoAvantiSensor6_EMG1_Normalized	0.659918072	
TrignoAvantiSensor14_EMG1_Normalized	0.636298858	

Figure 25-Hurst Component for dominant channels of pinky

For the pinky gesture, Channel 6 and Channel 14 yield Hurst exponents of **0.6599** and **0.6363**, respectively. These values also indicate persistent behavior, though slightly lower than those observed for the index gesture. This suggests that pinky activation exhibits greater variability and less sustained recruitment compared to index finger motion, which is consistent with the finer and less forceful nature of pinky movements.

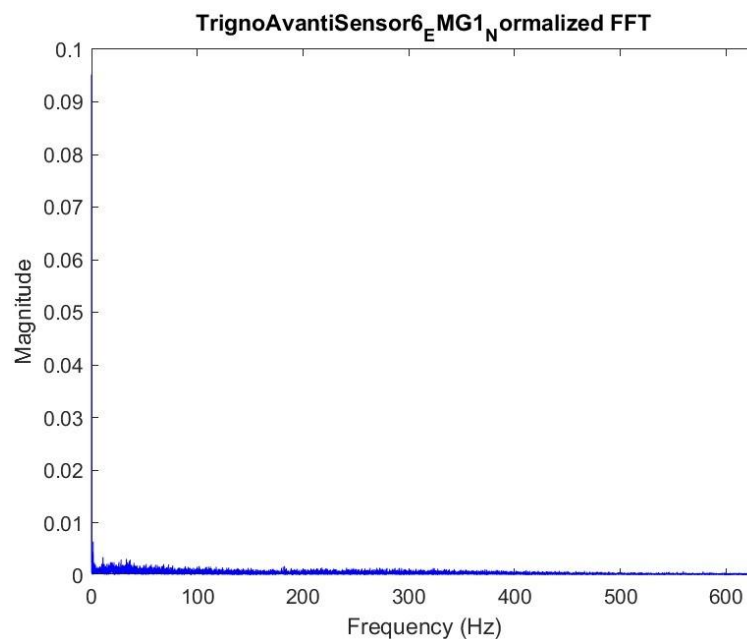


Figure 26- FFT for channel 6 of pinky finger

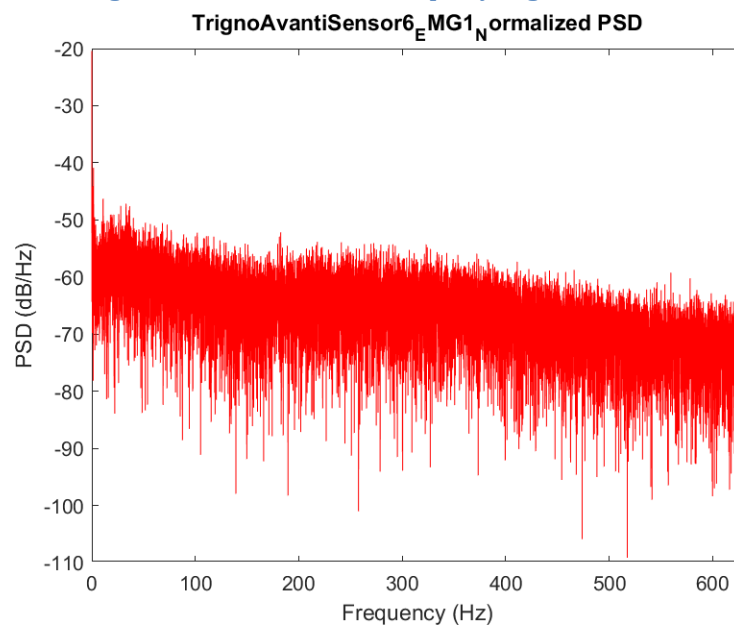


Figure 27- PSD for channel 6 of pinky finger

A	B
Sensor	PSD_BandPower_20_450Hz
TrignoAvantiSensor14_EMG1_Normalized	0.009057764
TrignoAvantiSensor6_EMG1_Normalized	0.007452708

Figure 28-PSD band power 20-450Hz for pinky finger

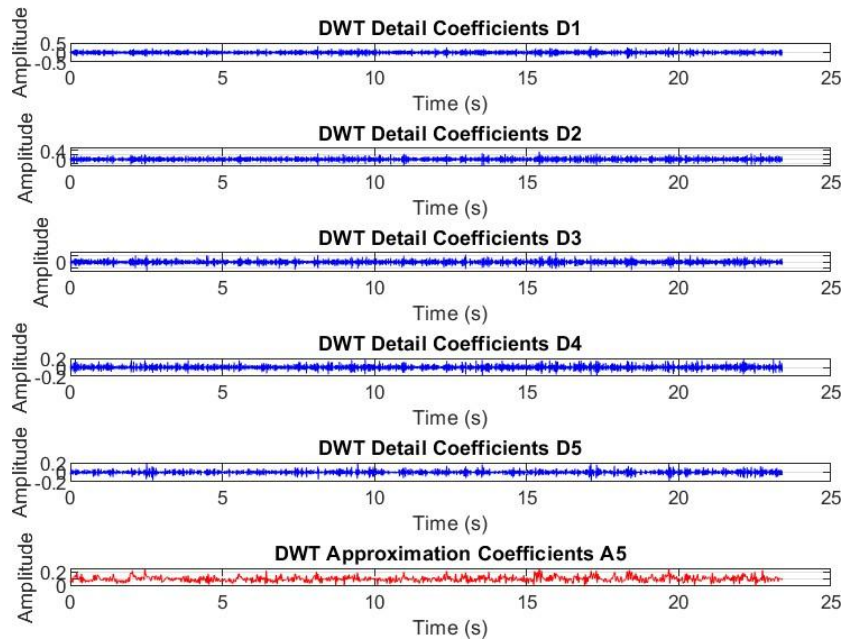


Figure 29- DWT for channel 6 of pinky finger

P	Q	R	S
TrignoAvantiSensor6_EMG1_Normalized_D3D4_Sum	TrignoAvantiSensor14_EMG1_Normalized_D3D4_Sum	Channels	Total Summation of absolute D3,D4
-0.06253293	-0.017919523	6	1842
-0.06253293	-0.017919523	14	528
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		
-0.06253293	-0.017919523		

Figure 30- Summation for absolute D3 & D4 for pinky dominant channels

The noticeably higher total summation of the absolute D3–D4 values for Channel 6 compared to Channel 14 indicates a stronger overall muscle activation or higher signal energy within the D3–D4 frequency band during the recording period.

CAD implementation and testing using a dummy signal on MATLAB

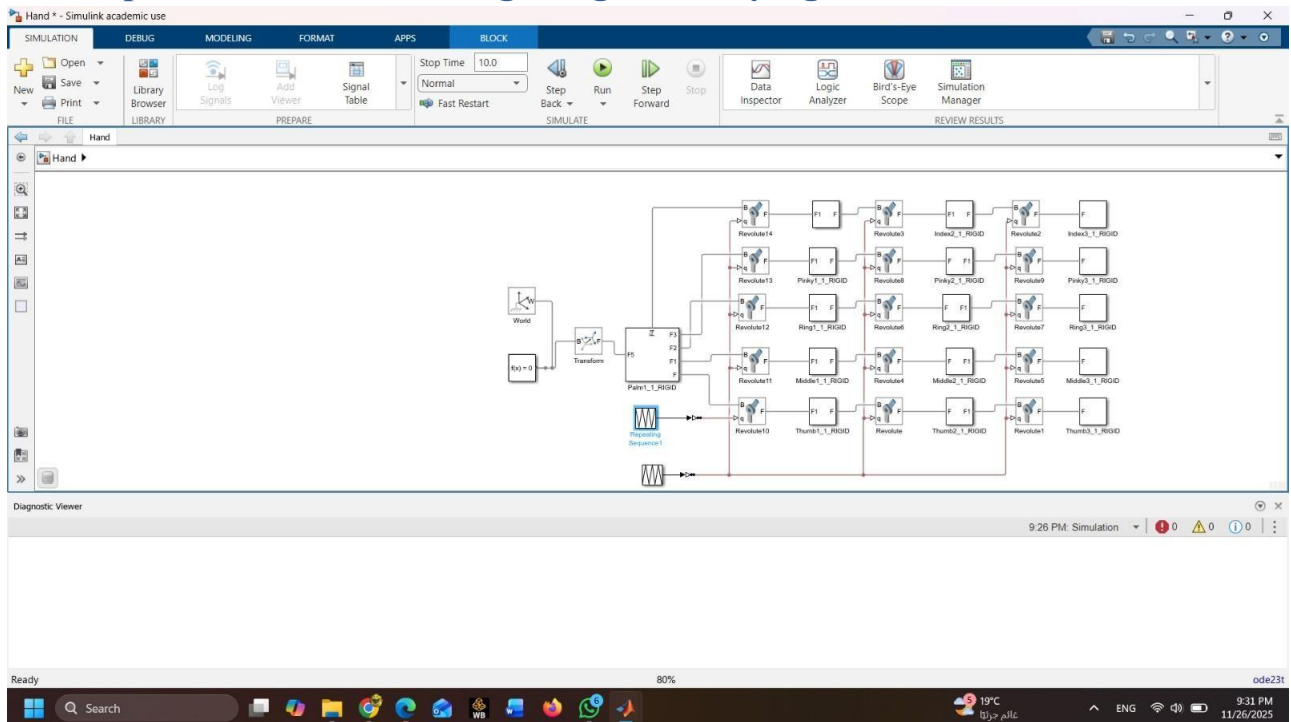


Figure 31- Snippet of MATLAB sim to test joints' motion

This Simulink model represents a prosthetic hand with multiple joints, where each finger and its segments are modeled using rigid bodies and revolute joints.

To summarize each key aspect of this simulation:

1. Base and Palm:

- The World block defines the global reference frame.
- Transform sets the initial position and orientation of the hand.
- Palm1_1_RIGID is the central rigid body connecting all fingers.

2. Fingers:

- Each finger (Thumb, Index, Middle, Ring, Pinky) is composed of three rigid segments (e.g., Index2_1_RIGID, Index3_1_RIGID) connected by revolute joints (Revolute blocks).
- The revolute joints allow rotation like natural finger bending.

3. Dummy Signal:

- The Repeating Sequence block acts as a dummy input signal to simulate joint actuation.
- This signal drives the revolute joints to test if the finger segments move correctly.

4. Purpose:

- This setup is used to verify joint motion without connecting to real EMG or sensor inputs.
- It ensures that each finger segment responds to control signals and the kinematic chain works properly.

5. Outputs:

- Each joint's motion can be visualized using Rigid Transform outputs or scopes (not shown in this screenshot).
- By running the simulation, you can see the prosthetic hand flex and extend according to the dummy input.

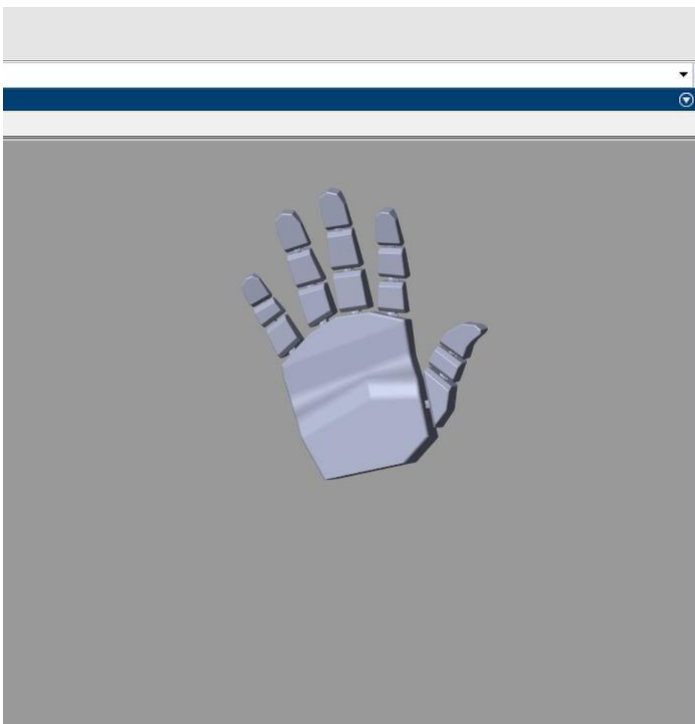


Figure 32- Prosthetic hand relaxed



Figure 33- Prosthetic hand contracted

7. Flowchart for Control Algorithm

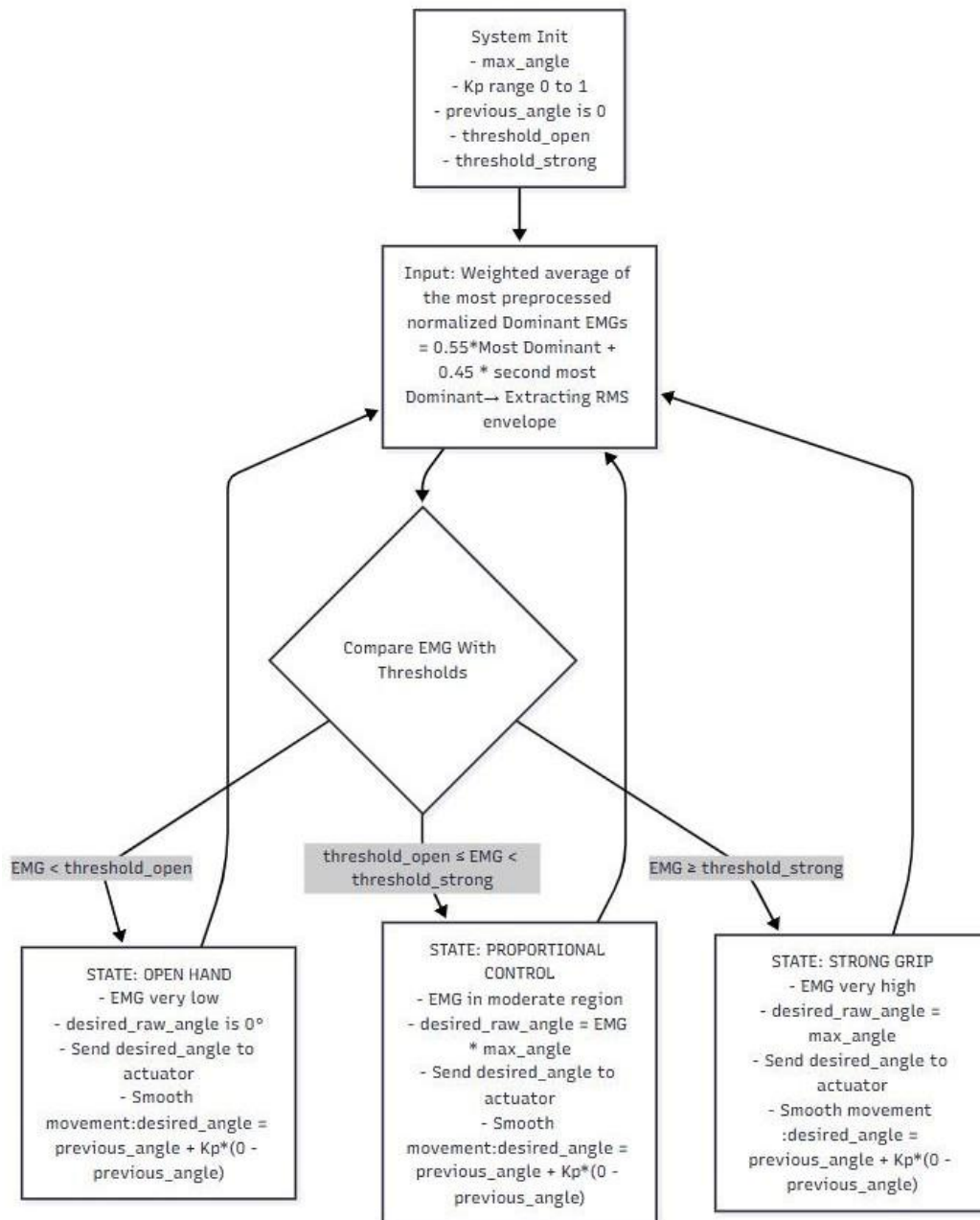


Figure 34- flowchart for control algorithm

7.1. Control Algorithm Description

The control algorithm translates processed electromyography (EMG) signals into controlled, smooth movements of a prosthetic hand. The system operates continuously in real time and adjusts the hand's position based on how much the user activates their muscles.

7.1.1. System Initialization

At the start of operation, the system initializes all essential parameters required for stable control:



- **Maximum joint angle (max_angle):** Defines the maximum allowable flexion angle of the prosthetic fingers.
- **Proportional gain (Kp):** Limited to the range 0–1 to ensure smooth and stable motion without overshoot.
- **Previous angle (previous_angle):** Initialized to zero to provide a known starting point for motion smoothing.
- **Threshold values:**
 - a) threshold_open: Minimum EMG level required to initiate hand closing.
 - b) threshold_strong: EMG level corresponding to strong muscle contraction and full grip.

This initialization stage ensures repeatable and predictable system behavior.

7.1.2. EMG Signal Input and Processing

The control input is derived from **two dominant EMG channels**, selected during preprocessing based on signal quality and relevance.

- The EMG signals are **normalized** to ensure consistent scaling.
- A **weighted average** is computed to improve robustness:

$$EMG_{combined} = 0.55 \times EMG_{dominant} + 0.45 \times EMG_{second\ dominant}$$

- The RMS envelope of the combined EMG signal is extracted to represent muscle activation intensity while reducing noise and high-frequency variations.

This processed EMG envelope is used as the main control variable for decision-making.

7.1.3. Threshold Comparison and State Selection

The processed EMG signal is continuously compared against predefined thresholds to determine the appropriate control state:

7.1.3.1. Open Hand State

Condition:

$$EMG < threshold_{open}$$

- Muscle activity is minimal.
- The system interprets this as a command to open the hand.



- The desired raw joint angle is set to 0° .

7.1.3.2. Proportional Control State

Condition:

$$threshold_{open} \leq EMG < threshold_{strong}$$

- Muscle activation is moderate.
- The desired joint angle is computed proportionally:

$$desired_raw_angle = EMG \times max_angle$$

- This allows continuous and intuitive control of finger flexion based on muscle effort.

c) Strong Grip State

Condition:

$$EMG \geq threshold_{strong}$$

- Muscle activation is very high.
- The system commands a **full grip**, setting:

$$desired_raw_angle = max_angle$$

This state-based logic ensures clear separation between rest, gradual motion, and maximum grip force.

7.1.4. Motion Smoothing and Actuation

To prevent abrupt or jerky motion, a proportional smoothing mechanism is applied in all states before sending commands to the actuator:

$$desired_angle = previous_angle + K_p(desired_raw_angle - previous_angle)$$

Where:

- `previous_angle` is the joint angle from the last control cycle.
- K_p controls the rate of convergence toward the desired angle.

This ensures:

7.1.5. Continuous Feedback Loop

- Updating EMG inputs
- Re-evaluating thresholds
- Adjusting joint angles in real time

8. Final MATLAB model



32

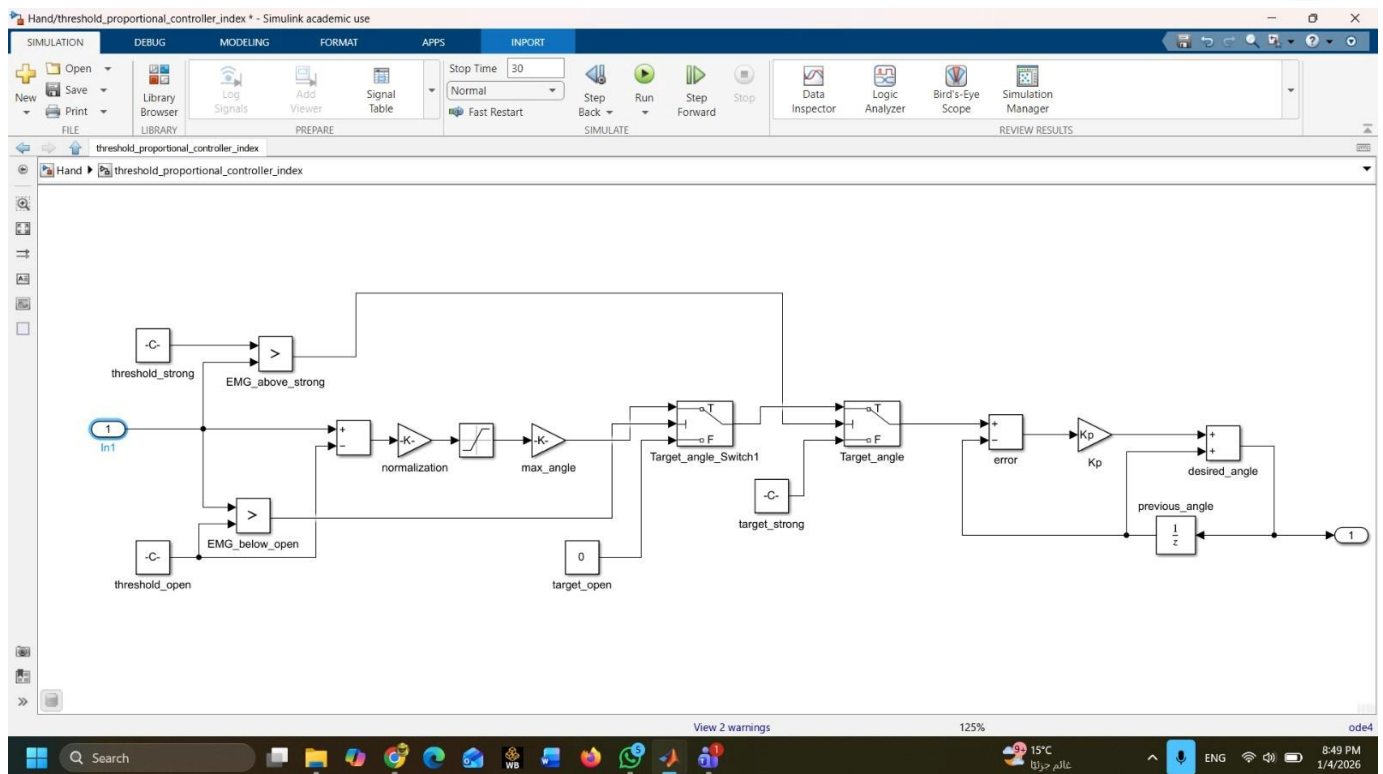


Figure 36- Controller subsystem model on MATLAB

This Simulink subsystem implements the control logic described in the flowchart, translating the processed EMG signal into a smooth desired joint angle for the prosthetic finger. The controller combines threshold-based decision making, proportional angle mapping, and first-order smoothing.

1. EMG Input

- **In1** represents the processed and normalized EMG envelope obtained after preprocessing (filtering, rectification, and RMS extraction).
- This signal serves as the main control input to the subsystem.

2. Threshold Comparison Logic

Two relational operator blocks are used to classify the EMG signal into control regions:

i. Strong Contraction Detection

- The EMG signal is compared with `threshold_strong` using a **greater-than (>)** block.
- Output signal: **EMG_above_strong**
 - Logical 1 when $EMG \geq threshold_strong$
 - Logical 0 otherwise

This signal is used to activate the strong grip **state**.



ii. Open Hand Detection

- The EMG signal is compared with threshold_open using another greater-than (>) block.
- Output signal: EMG_below_open
 - Indicates minimal muscle activity

This signal enables the open hand state.

3. Proportional Angle Generation Path

When the EMG lies between the two thresholds, the controller operates in **proportional mode**:

1. Normalization Block

- Scales the EMG signal to a normalized range (typically 0–1).
- Ensures consistent behavior across users and sessions.

2. Saturation Block

- Prevents the signal from exceeding physical or logical bounds.
- Protects against excessive joint commands.

3. max_angle Gain

- Converts the normalized EMG value into a physical joint angle:

$$\theta_{prop} = EMG_{norm} \times \theta_{max}$$

This represents the desired joint angle in proportional control mode.

4. Target Angle Selection (State Logic)

Two **Switch** blocks implement the state-based behavior of the controller:

a) First Switch – Strong Grip Override

- If EMG_above_strong = 1:
 - ❖ The output is forced to target_strong (equal to max_angle)
- Otherwise:
 - ❖ The proportional angle is passed forward

This guarantees full finger closure during strong muscle contractions.



b) Second Switch – Open Hand Override

- If $EMG_below_open = 1$:
 - ❖ The output is forced to $target_open = 0$
- Otherwise:
 - ❖ The output from the first switch is passed through

This ensures that the hand fully opens when muscle activity is low.

5. Error Computation

- The selected **target angle** is compared with the **previous joint angle**:

$$error = \theta_{target} - \theta_{previous}$$

This error represents how far the joint is from the desired position.

6. Proportional Smoothing Control

- The error is multiplied by the proportional gain K_p .
- The scaled correction is added to the previous angle:

$$\theta_{desired} = \theta_{previous} + K_p(\theta_{target} - \theta_{previous})$$

This implements a first-order low-pass behavior, ensuring:

- Smooth motion
- Reduced mechanical stress
- No jerky joint movements

7. Memory Block (State Feedback)

- A Unit Delay ($1/z$) block stores the current output angle.
- This value becomes $previous_angle$ for the next control cycle.

This feedback loop enables continuous and stable real-time control.



8. Output

- The final signal desired_angle is sent to the actuator or joint controller.
- It represents a smooth, physically feasible joint angle command

9. Conclusion

This work demonstrates a structured and reliable approach for transforming raw EMG data into meaningful control signals for a hand motion simulation. Starting with data conversion and preprocessing, each stage of the pipeline contributes to improving signal quality and interpretability. The combined use of time-domain, frequency-domain, and time–frequency analyses provides a comprehensive characterization of muscle activity and enables informed feature selection.

The identification of dominant channels based on IEMG and RMS proved effective in isolating gesture-relevant muscle activity for both index and pinky movements. The weighted averaging of dominant channels, followed by envelope extraction, resulted in smooth and robust control inputs suitable for real-time simulation. The MATLAB/Simulink implementation validated the control strategy, with threshold-based state transitions and proportional control enabling stable, continuous hand motion within defined kinematic limits.

Overall, the results confirm that the proposed methodology successfully bridges EMG signal processing and biomechanical simulation. The framework is modular, extensible, and well-suited for future work involving real-time control, additional gestures, or integration with physical prosthetic or robotic hand systems.

10. Submission link

[MCT343 team 7 final submission](#)



11. Appendix

11.1. Conversion from .json to a structured .csv for better file handling

```
import json
import pandas as pd
import numpy as np

# -----
# STEP 1: Load the JSON file
# -----
with open('Dynamic_2.json', 'r') as file:
    data = json.load(file) # Load the JSON content as a Python dictionary

# -----
# STEP 2: Extract EMG (Analog) Data
# -----

# Find the "Analog" block where EMG and IMU sensor data are stored
analog_section = next(item for item in data['Analog'] if item['Name'] == 'Delsys Trigno API')

# Read frequency for time reconstruction (important for timestamping)
emg_frequency = analog_section['Timebase']['Frequency'] # e.g., 1259.26 Hz

# Read sensor labels (e.g., "Sensor1_EMG 1", "Sensor1_ACC X", etc.)
emg_labels = analog_section['Labels']

# Read the actual recorded values for each timestamp
emg_values = np.array(analog_section['Values']).T # Transpose to shape (samples, channels)

# Construct a time axis using sample index and known frequency
emg_time = np.arange(emg_values.shape[0]) / emg_frequency

# Create a DataFrame for easy handling and exporting
emg_df = pd.DataFrame(emg_values, columns=emg_labels)
emg_df.insert(0, 'Time', emg_time) # Insert time as the first column

# -----
# STEP 3: Extract Movement (Skeleton) Data
# -----

# Assume we only have one skeleton (usually true in single-subject recordings)
skeleton_data = data['Skeletons'][0]

# Get the sampling frequency for the movement data (likely 300 Hz)
skeleton_frequency = skeleton_data['Timebase']['Frequency']
```



```
# Access each body segment's movement data
segments = skeleton_data.get('Segments', []) # Example: RightHand, RightHandIndex1, etc.

# Dictionary to hold individual segment DataFrames
movement_dfs = {}

for segment in segments:
    segment_name = segment['Name'] # Example: 'RightHand'

    # Extract the position and rotation data (X, Y, Z + Quaternion x, y, z, w)
    values = segment['Parts'][0]['Values']
    values_array = np.array(values)

    # Build time axis for this segment
    time_axis = np.arange(values_array.shape[0]) / skeleton_frequency

    # Create DataFrame for this segment with proper column names
    df = pd.DataFrame(values_array, columns=[
        f'{segment_name}_X',
        f'{segment_name}_Y',
        f'{segment_name}_Z',
        f'{segment_name}_Qx',
        f'{segment_name}_Qy',
        f'{segment_name}_Qz',
        f'{segment_name}_Qw',
    ])
    df.insert(0, 'Time', time_axis) # Insert time as first column
    movement_dfs[segment_name] = df # Store in the dictionary

# Combine all segment DataFrames on their Time index
# This aligns all the movement data into one DataFrame
movement_df = pd.concat(
    [df.set_index('Time') for df in movement_dfs.values()],
    axis=1
).reset_index() # Reset index to make 'Time' a regular column

# -----
# STEP 4: Save Both DataFrames to CSV Files
# -----

# Save the EMG data with time column
emg_df.to_csv('emg_data.csv', index=False)

# Save the movement (skeleton) data with time column
movement_df.to_csv('movement_data.csv', index=False)

print("☑ EMG and movement data successfully extracted and saved as CSV files.")
```



11.2. Preprocessing MATLAB script

```
>> fs = 1259;
file = 'emg_data.csv';

% Load CSV file and read headers
opts = detectImportOptions(file);
headers = opts.VariableNames;

data = readmatrix(file);
time = data(:,1);
rawData = data(:,2:end);

% Find EMG channels only
emgCols = find(contains(headers, '_EMG'));
numEMG = length(emgCols);
disp(['Detected ', num2str(numEMG), ' EMG channels.']);

% Bandpass filter for EMG
lowCut = 10;
highCut = 500;
order = 4;
[b,a] = butter(order, [lowCut highCut]/(fs/2), 'bandpass');

% Apply filter to EMG channels
filteredData = rawData;
for i = 1:numEMG
    col = emgCols(i) - 1;
    filteredData(:,col) = filter(b, a, rawData(:,col));
end

% Full-wave rectification
rectifiedData = filteredData;
for i = 1:numEMG
    col = emgCols(i) - 1;
    rectifiedData(:,col) = abs(filteredData(:,col));
end

% Normalize each EMG channel
normalizedData = rectifiedData;
for i = 1:numEMG
    col = emgCols(i) - 1;
    maxVal = max(rectifiedData(:,col));
    if maxVal > 0
        normalizedData(:,col) = rectifiedData(:,col) / maxVal;
    end
end
```



```
% Plot raw EMG signals
figure('Name','Raw EMG Channels');
hold on; grid on;
for i = 1:numEMG
    col = emgCols(i) - 1;
    plot(time, rawData(:,col));
end
xlabel('Time (s)');
ylabel('Amplitude');
title('Raw EMG Signals');
legend(headers(emgCols));
saveas(gcf, 'RAW_EMG.png');
close;

% Plot filtered EMG
figure('Name','Filtered EMG Channels');
hold on; grid on;
for i = 1:numEMG
    col = emgCols(i) - 1;
    plot(time, filteredData(:,col));
end
xlabel('Time (s)');
ylabel('Amplitude');
title('Filtered EMG (10-500 Hz)');
legend(headers(emgCols));
saveas(gcf, 'Filtered_EMG.png');
close;

% Plot rectified EMG
figure('Name','Rectified EMG Channels');
hold on; grid on;
for i = 1:numEMG
    col = emgCols(i) - 1;
    plot(time, rectifiedData(:,col));
end
xlabel('Time (s)');
ylabel('Amplitude');
title('Rectified EMG');
legend(headers(emgCols));
saveas(gcf, 'Rectified_EMG.png');
close;
```




```
% Plot normalized EMG
figure('Name','Normalized EMG Channels');
hold on; grid on;
for i = 1:numEMG
    col = emgCols(i) - 1;
    plot(time, normalizedData(:,col));
end
xlabel('Time (s)');
ylabel('Normalized Amplitude');
title('Normalized EMG');
legend(headers(emgCols));
saveas(gcf, 'Normalized_EMG.png');
close;

% Save all results to a new CSV file
timeHeader = headers(1);
signalHeaders = headers(2:end);

output = array2table( ...
    [time rawData filteredData rectifiedData normalizedData], ...
    'VariableNames', [ ...
        timeHeader, ...
        signalHeaders, ...
        append(signalHeaders, '_Filtered'), ...
        append(signalHeaders, '_Rectified'), ...
        append(signalHeaders, '_Normalized') ...
    ] ...
);

writetable(output, 'emg_preprocessed.csv');

disp('Processing finished successfully.');
```

Detected 14 EMG channels.
Processing finished successfully.



11.3. Time-domain analyses script

```
>> % Parameters
filename = 'emg_preprocessed.csv';
Fs = 1259;
windowLength = 0.2;
overlapPerc = 0.7;
thresholdSSC = 0.01;

% Read CSV and Headers
opts = detectImportOptions(filename);
fid = fopen(filename);
headerLine = fgetl(fid);
fclose(fid);
headers = strsplit(headerLine, opts.Delimiter);

emgCols = find(contains(headers, '_EMG'));
numSensors = length(emgCols);

data = readmatrix(filename, 'Delimiter', opts.Delimiter, 'NumHeaderLines', 1);
timeVec = data(:,1);

% Window Setup
winSamples = round(windowLength * Fs);
stepSamples = round(winSamples * (1 - overlapPerc));
numSamples = length(timeVec);

startIdx = 1:stepSamples:(numSamples - winSamples + 1);
numWindows = length(startIdx);
windowCenters = startIdx + floor(winSamples / 2);

% Prepare Storage
featuresAll = zeros(numWindows, numSensors * 5);
featureNames = {'MAV', 'RMS', 'WL', 'iEMG', 'SSC'};
```



```
% Process Each EMG Channel
for s = 1:numSensors
    emgSignal = data(:, emgCols(s));

    MAV = zeros(numWindows,1);
    RMS = zeros(numWindows,1);
    WL  = zeros(numWindows,1);
    iEMG = zeros(numWindows,1);
    SSC = zeros(numWindows,1);

    for w = 1:numWindows
        idxRange = startIdx(w):(startIdx(w) + winSamples - 1);
        winData = emgSignal(idxRange);

        MAV(w) = mean(abs(winData));
        RMS(w) = sqrt(mean(winData.^2));
        WL(w)  = sum(abs(diff(winData)));
        iEMG(w) = sum(abs(winData));

        sscCount = 0;
        for k = 2:length(winData)-1
            diff1 = winData(k) - winData(k-1);
            diff2 = winData(k) - winData(k+1);
            if (diff1 * diff2 > thresholdSSC)
                sscCount = sscCount + 1;
            end
        end
        SSC(w) = sscCount;
    end

    colStart = (s-1)*5 + 1;
    featuresAll(:, colStart:colStart+4) = [MAV, RMS, WL, iEMG, SSC];
end
```



```
% Plot EMG signal with extracted features
figure('Name', headers{emgCols(s)}, 'NumberTitle', 'off');
plot(timeVec, emgSignal, 'k'); hold on;

emgRange = max(emgSignal) - min(emgSignal);
scalingFactor = emgRange / max(max([MAV, RMS, WL, iEMG, SSC]));

plot(timeVec(windowCenters), MAV * scalingFactor, 'b', 'LineWidth', 1.2);
plot(timeVec(windowCenters), RMS * scalingFactor, 'r', 'LineWidth', 1.2);
plot(timeVec(windowCenters), WL * scalingFactor, 'g', 'LineWidth', 1.2);
plot(timeVec(windowCenters), iEMG * scalingFactor, 'm', 'LineWidth', 1.2);
plot(timeVec(windowCenters), SSC * scalingFactor, 'c', 'LineWidth', 1.2);

xlabel('Time (s)');
ylabel('Amplitude');
title(['EMG and Time-Domain Features - ', headers{emgCols(s)}]);
legend('EMG', 'MAV', 'RMS', 'WL', 'iEMG', 'SSC');
grid on;

saveas(gcf, ['EMG_Features_', headers{emgCols(s)}, '.png']);
close(gcf);
end

% Export Features to CSV
csvHeaders = {};
for s = 1:numSensors
    for f = 1:5
        csvHeaders{end+1} = [headers{emgCols(s)}, '_', featureNames{f}];
    end
end

outputTable = array2table( ...
    [timeVec(windowCenters), featuresAll], ...
    'VariableNames', ['Time', csvHeaders] ...
);

writetable(outputTable, 'EMG_Features_Windowed.csv');

disp('Windowed features calculated, plotted, and saved.');
```

Windowed features calculated, plotted, and saved.



11.4. Frequency-domain analyses script

```
% Parameters

filename = 'emg_preprocessed.csv';

Fs = 1259;

dominanceLowHz = 20;

dominanceHighHz = 450;

dominanceRatio = 0.7;

% Read CSV headers

opts = detectImportOptions(filename);

fid = fopen(filename);

headerLine = fgetl(fid);

fclose(fid);

headers = strsplit(headerLine, opts.Delimiter);

% Identify EMG channels

emgCols = find(contains(headers, '_EMG'));

numSensors = length(emgCols);

% Load numeric data

data = readmatrix(filename, 'Delimiter', opts.Delimiter, 'NumHeaderLines', 1);

timeVec = data(:,1); %#ok<NASGU>

% Preallocate storage

hurstFeatures = zeros(numSensors,1);

bandPower = zeros(numSensors,1);

% PSD CSV preparation

psdMatrix = [];

psdHeaders = [{'Frequency_Hz'}, headers(emgCols)];
```



```
% PROCESS EACH EMG CHANNEL

for i = 1:numSensors

    emgSignal = data(:, emgCols(i));

    Nsig      = length(emgSignal);

    % FFT computation

    Y          = fft(emgSignal);

    fFFT       = Fs*(0:Nsig-1)/Nsig;

    magnitude = abs(Y)/Nsig;

    % Power Spectral Density

    P1 = P2(1:floor(Nsig/2)+1);

    P1(2:end-1) = 2 * P1(2:end-1);

    freqPSD = Fs*(0:floor(Nsig/2))/Nsig;

    P1dB     = 10*log10(P1 + eps);

    % Store PSD values

    if i == 1

        psdMatrix = zeros(length(P1), numSensors + 1);

        psdMatrix(:,1) = freqPSD;

    end

    psdMatrix(:, i+1) = P1;
```




```
% Band power in EMG range

bandIdx = (freqPSD >= dominanceLowHz) & (freqPSD <= dominanceHighHz);

bandPower(i) = sum(P1(bandIdx));


% Save FFT plot

fig = figure('Visible','off');

plot(fFFT, magnitude, 'b');

xlim([0 Fs/2]);

xlabel('Frequency (Hz)');

ylabel('Magnitude');

title([headers{emgCols(i)}, ' FFT']);

saveas(fig, ['FFT_', headers{emgCols(i)}, '.png']);

close(fig);
```



```
% Save PSD plot

fig = figure('Visible','off');

plot(freqPSD, PldB, 'r');

xlim([0 Fs/2]);

xlabel('Frequency (Hz)');

ylabel('PSD (dB/Hz)');

title([headers{emgCols(i)}, ' PSD']);

saveas(fig, ['PSD_', headers{emgCols(i)}, '.png']);

close(fig);

% Hurst exponent estimation

X = cumsum(emgSignal - mean(emgSignal));

R = max(X) - min(X);

S = std(emgSignal);

if S ~= 0

    hurstFeatures(i) = log(R / S) / log(Nsig);
```



```

if S ~= 0

    hurstFeatures(i) = log(R / S) / log(Nsig);

else

    hurstFeatures(i) = 0;

end

end

% DOMINANT CHANNEL DETECTION

maxPower = max(bandPower);

dominantIdx = find(bandPower >= dominanceRatio * maxPower);

dominantChannels = headers(emgCols(dominantIdx));

disp('Dominant EMG Channels:');

disp(dominantChannels);

% Hurst exponent CSV

hurstHeader = {'Sensor','Hurst'};

hurstOutput = [headers(emgCols)', num2cell(hurstFeatures)];

hurstOutput = [hurstHeader; hurstOutput];

writecell(hurstOutput, 'EMG_Hurst.csv');

% PSD CSV

psdCell = [psdHeaders; num2cell(psdMatrix)];

writecell(psdCell, 'EMG_PSD.csv');

% Band power CSV

bandPowerHeader = {'Sensor','PSD_BandPower_20_450Hz'};

bandPowerOutput = [headers(emgCols)', num2cell(bandPower)];

bandPowerOutput = [bandPowerHeader; bandPowerOutput];

writecell(bandPowerOutput, 'EMG_PSD_BandPower.csv');

```



11.5. Frequency-time analysis script

```
%% ===== User Parameters =====
filename = 'emg_preprocessed.csv'; % Processed EMG CSV file
Fs = 1259; % Sampling frequency (Hz)
wname = 'db4'; % Wavelet type
level = 5; % Decomposition level

%% Read CSV and headers
opts = detectImportOptions(filename);
fid = fopen(filename);
headerLine = fgetl(fid);
fclose(fid);
headers = strsplit(headerLine, opts.Delimiter);

emgCols = find(contains(headers, '_EMG'));
numSensors = length(emgCols);

data = readmatrix(filename, 'Delimiter', opts.Delimiter, 'NumHeaderLines', 1);
timeVec = data(:,1);

coeffMatrix = zeros(length(timeVec), numSensors*(level+1) + 1);
coeffMatrix(:,1) = timeVec;
coeffHeaders = {'Time'};

sumD3 = zeros(1, numSensors);
sumD4 = zeros(1, numSensors);
sumD3D4 = zeros(1, numSensors);

%% ===== Process Each EMG Channel =====
for i = 1:numSensors
    emgSignal = data(:, emgCols(i));

    [C, L] = wavedec(emgSignal, level, wname);

    D_full = zeros(length(emgSignal), level);
    for j = 1:level
        D_full(:,j) = wrcoef('d', C, L, wname, j);
        coeffHeaders{end+1} = [headers{emgCols(i)}, '_D', num2str(j)];
    end
end
```



```

A_full = wrcoef('a', C, L, wname, level);
coeffHeaders{end+1} = [headers{emgCols(i)}, '_A', num2str(level)];

colStart = (i-1)*(level+1) + 2;
coeffMatrix(:, colStart:colStart+level) = [D_full, A_full];

sumD3(i) = sum(D_full(:,3));
sumD4(i) = sum(D_full(:,4));
sumD3D4(i) = sumD3(i) + sumD4(i);

figure('Name', headers{emgCols(i)}, 'NumberTitle', 'off');
for j = 1:level
    subplot(level+1, 1, j);
    plot(timeVec, D_full(:,j), 'b');
    title(['Detail Coefficient D', num2str(j)]);
    xlabel('Time (s)');
    ylabel('Amplitude');
    grid on;
end

subplot(level+1, 1, level+1);
plot(timeVec, A_full, 'r');
title(['Approximation Coefficient A', num2str(level)]);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

saveas(gcf, ['DWT_', headers{emgCols(i)}, '.png']);
close(gcf);
end

%% Append Sum Columns
for i = 1:numSensors
    coeffMatrix = [coeffMatrix, repmat(sumD3(i), length(timeVec), 1)];
    coeffHeaders{end+1} = [headers{emgCols(i)}, '_D3_Sum'];

    coeffMatrix = [coeffMatrix, repmat(sumD4(i), length(timeVec), 1)];
    coeffHeaders{end+1} = [headers{emgCols(i)}, '_D4_Sum'];

    coeffMatrix = [coeffMatrix, repmat(sumD3D4(i), length(timeVec), 1)];
    coeffHeaders{end+1} = [headers{emgCols(i)}, '_D3D4_Sum'];
end

%% Export Combined CSV

fid = fopen('EMG_DWT_Coefficients.csv', 'w');
fprintf(fid, '%s', coeffHeaders{1:end-1});
fprintf(fid, '%s\n', coeffHeaders{end});
fclose(fid);

writematrix(coeffMatrix, 'EMG_DWT_Coefficients.csv', 'WriteMode', 'append');

disp('All DWT figures saved and combined CSV created successfully.');
```



11.6. RMS envelope for weighted averaged signal

```

fs = 1259; % Sampling frequency
file = 'emg_data.csv';

%% ===== LOAD DATA & HEADERS =====
opts = detectImportOptions(file);
headers = opts.VariableNames;

data = readmatrix(file);
time = data(:,1);
signals = data(:,2:end);

%% ===== FIND WEIGHTED AVERAGE COLUMN =====
targetName = 'Dominant Channels _Weighted Average';
colIdx = find(strcmp(headers, targetName));

if isempty(colIdx)
    error('Column "Dominant Channels _Weighted Average" not found.');
```

end

```

% Adjust index because signals excludes time column
signalCol = colIdx - 1;
inputSignal = signals(:, signalCol);

disp('✓ Weighted average signal found.');
```

%% ===== RMS ENVELOPE (50 ms WINDOW) =====

```

window_ms = 50;
windowSamples = round((window_ms/1000) * fs);

rmsEnvelope = sqrt(movmean(inputSignal.^2, windowSamples));

%% ===== PLOT RMS ENVELOPE =====
figure('Name','RMS Envelope - Dominant Channels Weighted Average');
plot(time, rmsEnvelope,'LineWidth',1.5);
grid on;
xlabel('Time (s)');
ylabel('RMS Amplitude');
title('RMS Envelope (50 ms) - Dominant Channels Weighted Average');

saveas(gcf,'RMS_Envelope_Dominant_Weighted_Average.png');
close;
```

11.7. Average dominant channels:

```

%% ===== EXPORT CSV =====
output = array2table([time rmsEnvelope], ...
    'VariableNames', {'Time','Dominant_Weighted_RMS'});

writetable(output,'dominant_weighted_rms.csv');

disp('✓ RMS envelope computed.');
```

disp('✓ Plot saved: RMS_Envelope_Dominant_Weighted_Average.png');

```

disp('✓ CSV exported: dominant_weighted_rms.csv');
```




11.8. Hurst Component Formulas for calculations

Let $x(t)$, $t = 1, 2, \dots, N$, be a discrete time series for EMG signal.

Step 1: Compute the mean

$$\bar{x} = \frac{1}{N} \sum_{t=1}^N x(t)$$

Step 2: Form the cumulative deviation series

$$Y(t) = \sum_{i=1}^t (x(i) - \bar{x}), \quad t = 1, 2, \dots, N$$

Step 3: Compute the range $R(N)$

$$R(N) = \max_{1 \leq t \leq N} Y(t) - \min_{1 \leq t \leq N} Y(t)$$

Step 4: Compute the standard deviation $S(N)$

$$S(N) = \sqrt{\frac{1}{N} \sum_{t=1}^N (x(t) - \bar{x})^2}$$

Step 5: Rescaled range relationship

$$\frac{R(N)}{S(N)} = c N^H$$

where:

- H is the Hurst exponent
- c is a constant

Taking logarithms:

$$\log \left(\frac{R(N)}{S(N)} \right) = H \log (N) + \log (c)$$

The **slope** of the linear fit between $\log (R/S)$ and $\log (N)$ gives the Hurst exponent H .